Package ‘EstemPMM’

March 20, 2026

Type Package

Title Polynomial Maximization Method for Non-Gaussian Regression

Version 0.3.0

Date 2026-03-19

Description Implements the Polynomial Maximization Method ('PMM') for parameter estimation in linear and time series models when error distributions deviate from normality. The 'PMM2' variant achieves lower variance parameter estimates compared to ordinary least squares ('OLS') when errors exhibit significant skewness. The 'PMM3' variant (S=3) targets symmetric platykurtic error distributions, reducing variance when excess kurtosis is negative. Includes automatic method selection ('pmm_dispatch'), linear regression, 'AR'/'MA'/'ARMA'/'ARIMA' models, and bootstrap inference. Methodology described in Zabolotnii, Warsza, and Tkachenko (2018) <doi:10.1007/978-3-319-77179-3_75>, Zabolotnii, Tkachenko, and Warsza (2022) <doi:10.1007/978-3-031-03502-9_37>, and Zabolotnii, Tkachenko, and Warsza (2023) <doi:10.1007/978-3-031-25844-2_21>.

License GPL-3

Encoding UTF-8

Depends R (>= 3.5.0)

Imports methods, stats, graphics, utils

Suggests dplyr, ggplot2, gridExtra, testthat (>= 3.0.0), rmarkdown, knitr, MASS, numDeriv

LazyData true

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Config/testthat/edition 3

URL https://github.com/SZabolotnii/EstemPMM

BugReports https://github.com/SZabolotnii/EstemPMM/issues

VignetteBuilder knitr

NeedsCompilation no

Author Serhii Zabolotnii [aut, cre] (ORCID: <https://orcid.org/0000-0003-0242-2234>)

Maintainer Serhii Zabolotnii <zabolotniua@gmail.com>

Contents

EstemPMM-package . . . 4
.g2_factor . . . 6
.g3_factor . . . 6
.pmm2_fit . . . 7
.pmm3_nr_solver . . . 8
.pmm3_nr_solver_nonlinear . . . 8
.residual_cumulants . . . 9
.ts_pmm2_fit . . . 10
AIC,PMM2fit-method . . . 11
AIC,PMM3fit-method . . . 11
AIC,TS3fit-method . . . 12
ARIMAPMM2-class . . . 12
ARIMAPMM3-class . . . 12
arima_pmm2 . . . 13
arima_pmm3 . . . 15
ARMAPMM2-class . . . 16
ARMAPMM3-class . . . 16
arma_pmm2 . . . 16
arma_pmm3 . . . 18
ARPMM2-class . . . 19
ARPMM3-class . . . 19
ar_pmm2 . . . 19
ar_pmm3 . . . 21
auto_mpg . . . 23
BasePMM2-class . . . 24
coef,PMM2fit-method . . . 24
coef,PMM3fit-method . . . 25
coef,SARPMM2-method . . . 25
coef,SMAPMM2-method . . . 26
coef,TS2fit-method . . . 26
coef,TS3fit-method . . . 27
compare_arima_methods . . . 27
compare_arma_methods . . . 28
compare_ar_methods . . . 28
compare_ma_methods . . . 29
compare_sar_methods . . . 29
compare_ts_methods . . . 30
compare_with_ols . . . 31
compute_moments . . . 31
compute_moments_pmm3 . . . 32
compute_numerical_jacobian . . . 32
compute_pmm2_components . . . 33
compute_ts_residuals . . . 34
create_ar_matrix . . . 34
create_residual_function . . . 35
create_sarma_matrix . . . 35
create_sar_matrix . . . 37
create_ts_design_matrix . . . 38
DCOILWTICO . . . 38
djia2002 . . . 39

estpmm_style_ma . . . 40
estpmm_style_ma_sma . . . 40
estpmm_style_sma . . . 41
fitted,PMM2fit-method . . . 42
fitted,PMM3fit-method . . . 42
fitted,TS2fit-method . . . 43
fitted,TS3fit-method . . . 43
fitted_values . . . 44
get_ar_fitted . . . 44
get_classical_estimates . . . 45
get_initial_estimates . . . 45
get_sarimax_jacobian . . . 46
get_sarimax_residuals . . . 46
get_yw_estimates . . . 47
linearized_pmm2_wrapper . . . 47
lm_pmm2 . . . 48
lm_pmm3 . . . 49
MAPMM2-class . . . 51
MAPMM3-class . . . 51
ma_build_design . . . 51
ma_compute_innovations . . . 52
ma_pmm2 . . . 52
ma_pmm3 . . . 54
ma_sma_build_design . . . 55
ma_sma_compute_innovations . . . 56
ma_solve_pmm2 . . . 56
optimized_direct_pmm2 . . . 57
plot,PMM2fit,missing-method . . . 59
plot,PMM3fit,missing-method . . . 59
plot,TS2fit,missing-method . . . 60
plot,TS3fit,missing-method . . . 60
plot_pmm2_bootstrap . . . 61
PMM2fit-class . . . 61
pmm2_algorithm . . . 62
pmm2_inference . . . 63
pmm2_monte_carlo_compare . . . 63
pmm2_nonlinear_iterative . . . 65
pmm2_nonlinear_onestep . . . 65
pmm2_variance_factor . . . 66
pmm2_variance_matrices . . . 66
PMM3fit-class . . . 67
pmm3_variance_factor . . . 67
pmm_dispatch . . . 68
pmm_gamma6 . . . 69
pmm_kurtosis . . . 69
pmm_skewness . . . 70
predict,PMM2fit-method . . . 70
predict,PMM3fit-method . . . 71
predict,TS2fit-method . . . 71
predict,TS3fit-method . . . 72
residuals,PMM2fit-method . . . 72
residuals,PMM3fit-method . . . 73

residuals,TS2fit-method . . . 73
residuals,TS3fit-method . . . 74
SARIMAPMM2-class . . . 74
sarima_pmm2 . . . 75
SARMAPMM2-class . . . 77
sarma_pmm2 . . . 78
SARPMM2-class . . . 80
sar_pmm2 . . . 81
SMAPMM2-class . . . 83
sma_build_design . . . 84
sma_compute_innovations . . . 84
sma_css_fit . . . 85
sma_pmm2 . . . 85
sma_pmm2_fit . . . 87
solve_pmm2 . . . 88
solve_pmm2_step . . . 89
summary,PMM2fit-method . . . 89
summary,PMM3fit-method . . . 90
summary,SARIMAPMM2-method . . . 90
summary,SARMAPMM2-method . . . 91
summary,SARPMM2-method . . . 91
summary,SMAPMM2-method . . . 92
summary,TS2fit-method . . . 92
summary,TS3fit-method . . . 93
test_symmetry . . . 93
TS2fit-class . . . 94
TS3fit-class . . . 95
ts_pmm2 . . . 95
ts_pmm2_inference . . . 96
ts_pmm3 . . . 97
unified_pmm2_wrapper . . . 98
update_ma_innovations . . . 99
validate_ts_parameters . . . 100

Index **101**

`EstemPMM-package` *EstemPMM: Polynomial Maximization Method for Robust Regression and Time Series*

Description

The EstemPMM package provides robust methods for estimating parameters of linear models and time series models that are robust to non-Gaussian errors.

Linear Regression Functions

`lm_pmm2` - Fit linear models using PMM2

`compare_with_ols` - Compare PMM2 with OLS

Time Series Functions

`ts_pmm2` - General function for fitting time series models using PMM2

`ar_pmm2` - Fit AR models

`ma_pmm2` - Fit MA models

`arma_pmm2` - Fit ARMA models

`arima_pmm2` - Fit ARIMA models

`compare_ts_methods` - Compare PMM2 with classical methods

Statistical Inference

`pmm2_inference` - Bootstrap inference for linear models

`ts_pmm2_inference` - Bootstrap inference for time series models

PMM3 (Symmetric Errors)

`lm_pmm3` - Fit linear models using PMM3 (S=3, symmetric platykurtic errors)

`compute_moments_pmm3` - Compute moments for PMM3

`pmm3_variance_factor` - PMM3 theoretical variance reduction factor

`pmm_gamma6` - Compute sixth-order cumulant coefficient

`test_symmetry` - Test residual symmetry for PMM2 vs PMM3 choice

PMM3 Time Series Functions

`ts_pmm3` - General function for fitting time series models using PMM3

`ar_pmm3` - Fit AR models using PMM3

`ma_pmm3` - Fit MA models using PMM3

`arma_pmm3` - Fit ARMA models using PMM3

`arima_pmm3` - Fit ARIMA models using PMM3

Method Selection

`pmm_dispatch` - Automatic PMM method selection (OLS / PMM2 / PMM3)

Utilities

`pmm_skewness` - Compute skewness

`pmm_kurtosis` - Compute kurtosis

`compute_moments` - Compute moments and cumulants

Author(s)

Maintainer: Serhii Zabolotnii <zabolotniua@gmail.com> (ORCID)

See Also

Useful links:

- https://github.com/SZabolotnii/EstemPMM
- Report bugs at https://github.com/SZabolotnii/EstemPMM/issues

.g2_factor *PMM2 theoretical efficiency factor*

Description

PMM2 theoretical efficiency factor

Usage

```
.g2_factor(gamma3, gamma4)
```

Arguments

gamma3	Skewness coefficient
gamma4	Excess kurtosis coefficient

Value

Numeric scalar: g2 factor

.g3_factor *PMM3 theoretical efficiency factor*

Description

PMM3 theoretical efficiency factor

Usage

```
.g3_factor(gamma4, gamma6)
```

Arguments

gamma4	Excess kurtosis coefficient
gamma6	Sixth-order cumulant coefficient

Value

Numeric scalar: g3 factor

.pmm2_fit — *PMM2 fitting algorithm - unified implementation*

Description

Core iterative algorithm for PMM2 parameter estimation

Usage

```
.pmm2_fit(
  b_init,
  X,
  y,
  m2,
  m3,
  m4,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`b_init`	initial parameter estimates (typically from OLS)
`X`	design matrix
`y`	response vector
`m2, m3, m4`	central moments
`max_iter`	maximum number of iterations
`tol`	convergence tolerance
`regularize`	logical, add small value to diagonal for numerical stability
`reg_lambda`	regularization parameter (if regularize=TRUE)
`verbose`	logical, whether to print progress information

Value

A list with components:

`b`	estimated parameters
`convergence`	logical convergence status
`iterations`	number of iterations performed

.pmm3_nr_solver *PMM3 Newton-Raphson solver (vectorised)*

Description

Solves the PMM3 score equations using Newton-Raphson iteration. The score is $Z = X'(\varepsilon(\kappa - \varepsilon^2))$ and the Jacobian is $J = X'\mathrm{diag}(3\varepsilon^2 - \kappa)X$.

Usage

```
.pmm3_nr_solver(
  B_ols,
  X,
  Y,
  kappa,
  adaptive = FALSE,
  tol = 1e-06,
  max_iter = 100,
  step_max = 5
)
```

Arguments

B_ols	OLS starting values (length P)
X	Design matrix (n x P)
Y	Response vector (length n)
kappa	Moment ratio (scalar)
adaptive	Re-estimate kappa each iteration? (default FALSE)
tol	Convergence tolerance on ‖delta‖_2 (default 1e-6)
max_iter	Maximum NR iterations (default 100)
step_max	Maximum step size (default 5.0)

Value

List: B (estimates), converged, iter, kappa, near_gaussian

.pmm3_nr_solver_nonlinear *PMM3 Newton-Raphson solver for nonlinear models*

Description

Uses a residual function and numerical Jacobian (via numDeriv) instead of a fixed design matrix. This is necessary for ARIMA and other models where the linearized design matrix does not capture full dynamics.

Usage

```
.pmm3_nr_solver_nonlinear(
  theta_init,
  fn_residuals,
  kappa,
  adaptive = FALSE,
  tol = 1e-06,
  max_iter = 100,
  step_max = 5
)
```

Arguments

`theta_init`	Initial parameter estimates (from MLE/CSS)
`fn_residuals`	Function(theta) returning residual vector
`kappa`	Moment ratio (scalar)
`adaptive`	Re-estimate kappa each iteration? (default FALSE)
`tol`	Convergence tolerance (default 1e-6)
`max_iter`	Maximum NR iterations (default 100)
`step_max`	Maximum step size (default 5.0)

Details

The PMM3 estimating equation is:

$$\sum_t \varepsilon_t(\kappa - \varepsilon_t^2) \cdot \frac{\partial \varepsilon_t}{\partial \theta} = 0$$

Value

List: B (estimates), converged, iter, kappa, near_gaussian

`.residual_cumulants` *Compute residual cumulants up to 6th order*

Description

Compute residual cumulants up to 6th order

Usage

```
.residual_cumulants(residuals)
```

Arguments

`residuals`	Numeric vector of residuals

Value

Named list: m2, m3, m4, m6, gamma3, gamma4, gamma6

`.ts_pmm2_fit` *PMM2 fitting algorithm for time series models*

Description

Implementation of PMM2 algorithm for time series models with various approaches depending on the model structure (AR, MA, ARMA, ARIMA)

Usage

```
.ts_pmm2_fit(
  b_init,
  innovations_init,
  model_info,
  m2,
  m3,
  m4,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`b_init`	initial parameter estimates (AR followed by MA)
`innovations_init`	initial innovations (errors)
`model_info`	list with model structure information
`m2`, `m3`, `m4`	central moments
`max_iter`	maximum number of iterations
`tol`	convergence tolerance
`regularize`	logical, add small value to diagonal for numerical stability
`reg_lambda`	regularization parameter (if regularize=TRUE)
`verbose`	logical, whether to print progress information

Value

A list with components:

`b`	estimated parameters
`convergence`	logical convergence status
`iterations`	number of iterations performed
`innovations`	final innovations (errors)

AIC,PMM2fit-method *Calculate AIC for PMM2fit object*

Description

Calculate AIC for PMM2fit object

Usage

```
## S4 method for signature 'PMM2fit'
AIC(object, ..., k = 2)
```

Arguments

`object`	PMM2fit object
`...`	Additional arguments (not used)
`k`	Penalty per parameter to be used; default is 2

Value

AIC value

AIC,PMM3fit-method *Calculate AIC for PMM3fit object*

Description

Calculate AIC for PMM3fit object

Usage

```
## S4 method for signature 'PMM3fit'
AIC(object, ..., k = 2)
```

Arguments

`object`	PMM3fit object
`...`	Additional arguments (not used)
`k`	Penalty per parameter (default 2)

Value

AIC value

AIC,TS3fit-method *Calculate AIC for TS3fit object*

Description

Calculate AIC for TS3fit object

Usage

```
## S4 method for signature 'TS3fit'
AIC(object, ..., k = 2)
```

Arguments

object	TS3fit object
...	Additional arguments (not used)
k	Penalty per parameter (default 2)

Value

AIC value

ARIMAPMM2-class *S4 class for storing PMM2 ARIMA model results*

Description

S4 class for storing PMM2 ARIMA model results

ARIMAPMM3-class *S4 class for PMM3 ARIMA model results*

Description

S4 class for PMM3 ARIMA model results

arima_pmm2 *Fit an ARIMA model using PMM2 (wrapper)*

Description

Estimates autoregressive integrated moving-average model parameters using PMM2. ARIMA models extend ARMA to non-stationary series via differencing. PMM2 provides robust parameter estimates for the stationary ARMA component after differencing is applied.

Usage

```
arima_pmm2(
  x,
  order = c(1, 1, 1),
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification: - For AR models: a single integer (AR order) - For MA models: a single integer (MA order) - For ARMA models: vector c(p, q) (AR and MA orders) - For ARIMA models: vector c(p, d, q) (AR, differencing, and MA orders)
`method`	String: estimation method, one of "pmm2" (default), "css", "ml", "yw", "ols"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (specialized for MA/SMA models).
`max_iter`	Integer: maximum number of iterations for the algorithm
`tol`	Numeric: tolerance for convergence
`include.mean`	Logical: whether to include a mean (intercept) term
`initial`	List or vector of initial parameter estimates (optional)
`na.action`	Function for handling missing values, default is na.fail
`regularize`	Logical, add small values to diagonal for numerical stability
`reg_lambda`	Regularization parameter (if regularize=TRUE)
`verbose`	Logical: whether to print progress information

Details

PMM2 Variants:

- "unified_global" (default): One-step correction from MLE/CSS estimates. Fast and reliable for most ARIMA specifications.
- "unified_iterative": Full Newton-Raphson optimization. Recommended for ARIMA models with complex dynamics or when unified_global shows residual non-Gaussianity.
- "linearized": First-order approximation. Not typically recommended for ARIMA unless MA component dominates.

Variant Selection Guidelines:

- For standard ARIMA(p,d,q): Use "unified_global" (default)
- For models with d >= 2 or high orders: Try "unified_iterative"
- If MA component is large relative to AR: Consider "unified_iterative"

ARIMA Estimation Workflow:

1. Apply differencing of order d to achieve stationarity
2. Estimate ARMA(p,q) model on differenced series using PMM2
3. Return coefficients and diagnostics for the integrated model

Differencing Notes: The d parameter determines how many times the series is differenced. d=0 reduces to ARMA, d=1 handles unit root processes, d=2 is rare but useful for some economic series with trend acceleration.

Value

An S4 object of class ARIMAPMM2 containing fitted AR and MA coefficients, residual series, central moments, differencing order, intercept, original series, and convergence diagnostics.

See Also

arma_pmm2, sarima_pmm2, ar_pmm2

Examples

```
# Fit ARIMA(1,1,1) model to non-stationary series
x <- cumsum(arima.sim(n = 200, list(ar = 0.6, ma = -0.4)))
fit1 <- arima_pmm2(x, order = c(1, 1, 1))
coef(fit1)

# ARIMA(2,1,0) - random walk with AR(2) innovations
x2 <- cumsum(arima.sim(n = 250, list(ar = c(0.7, -0.3))))
fit2 <- arima_pmm2(x2, order = c(2, 1, 0), pmm2_variant = "unified_global")

# ARIMA(0,2,2) - double differencing with MA(2)
x3 <- cumsum(cumsum(arima.sim(n = 300, list(ma = c(0.5, 0.3)))))
fit3 <- arima_pmm2(x3, order = c(0, 2, 2), pmm2_variant = "unified_iterative")
```

arima_pmm3 *Fit an ARIMA model using PMM3*

Description

Estimates ARIMA model parameters using PMM3.

Usage

```
arima_pmm3(
  x,
  order = c(1, 1, 1),
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

x	Numeric vector of time series data
order	Numeric vector c(p, d, q): AR, differencing, and MA orders
max_iter	Integer: maximum NR iterations (default 100)
tol	Numeric: convergence tolerance (default 1e-6)
adaptive	Logical: re-estimate kappa each iteration (default FALSE)
step_max	Numeric: maximum NR step size (default 5.0)
include.mean	Logical: include mean term (default TRUE)
initial	Optional initial parameter estimates
na.action	Function for handling missing values (default na.fail)
verbose	Logical: print progress information (default FALSE)

Value

An S4 object of class ARIMAPMM3

See Also

arima_pmm2, arma_pmm3, lm_pmm3

Examples

```
set.seed(42)
x <- cumsum(arima.sim(n = 200, list(ar = 0.6),
            innov = runif(200, -sqrt(3), sqrt(3))))
fit <- arima_pmm3(x, order = c(1, 1, 0))
coef(fit)
```

ARMAPMM2-class *S4 class for storing PMM2 ARMA model results*

Description

S4 class for storing PMM2 ARMA model results

ARMAPMM3-class *S4 class for PMM3 ARMA model results*

Description

S4 class for PMM3 ARMA model results

---

arma_pmm2 *Fit an ARMA model using PMM2 (wrapper)*

Description

Estimates autoregressive moving-average model parameters using PMM2. ARMA models combine AR and MA components, capturing both direct past value dependencies and innovation structure. PMM2 leverages higher moments to improve parameter estimation accuracy.

Usage

```
arma_pmm2(
  x,
  order = c(1, 1),
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification: - For AR models: a single integer (AR order) - For MA models: a single integer (MA order) - For ARMA models: vector c(p, q) (AR and MA orders) - For ARIMA models: vector c(p, d, q) (AR, differencing, and MA orders)
`method`	String: estimation method, one of "pmm2" (default), "css", "ml", "yw", "ols"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (specialized for MA/SMA models).
`max_iter`	Integer: maximum number of iterations for the algorithm
`tol`	Numeric: tolerance for convergence
`include.mean`	Logical: whether to include a mean (intercept) term
`initial`	List or vector of initial parameter estimates (optional)
`na.action`	Function for handling missing values, default is na.fail
`regularize`	Logical, add small values to diagonal for numerical stability
`reg_lambda`	Regularization parameter (if regularize=TRUE)
`verbose`	Logical: whether to print progress information

Details

PMM2 Variants:

- `"unified_global"` (default): One-step correction from MLE/CSS estimates. Balances speed and accuracy for ARMA models. Recommended for most use cases.
- `"unified_iterative"`: Full Newton-Raphson optimization. Best for models with complex dynamics or strong non-Gaussian features.
- `"linearized"`: First-order approximation. Generally not recommended for ARMA; better suited for pure MA/SMA models.

Variant Selection Guidelines:

- For ARMA(p,q) with p,q <= 2: Use `"unified_global"` (default)
- For higher-order ARMA or ill-conditioned models: Try `"unified_iterative"`
- For quick exploration: Start with `"unified_global"`

ARMA Estimation Challenges: ARMA models have more parameters than pure AR or MA models, making estimation more sensitive to initialization and numerical stability. PMM2 benefits from robust moment-based constraints that help regularize the parameter space.

Value

An S4 object of class `ARMAPMM2` containing fitted AR and MA coefficients, residuals, central moments, model specification, intercept, original series, and convergence diagnostics.

See Also

`ar_pmm2`, `ma_pmm2`, `arima_pmm2`

Examples

```
# Fit ARMA(2,1) model
x <- arima.sim(n = 250, list(ar = c(0.7, -0.3), ma = 0.5))
fit1 <- arma_pmm2(x, order = c(2, 1))
coef(fit1)

# Try iterative variant for better accuracy
fit2 <- arma_pmm2(x, order = c(2, 1), pmm2_variant = "unified_iterative")

# Higher-order ARMA
x2 <- arima.sim(n = 300, list(ar = c(0.6, -0.2), ma = c(0.4, 0.3)))
fit3 <- arma_pmm2(x2, order = c(2, 2), pmm2_variant = "unified_global")
```

---

arma_pmm3 *Fit an ARMA model using PMM3*

Description

Estimates ARMA model parameters using PMM3.

Usage

```
arma_pmm3(
  x,
  order = c(1, 1),
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Numeric vector c(p, q): AR and MA orders
`max_iter`	Integer: maximum NR iterations (default 100)
`tol`	Numeric: convergence tolerance (default 1e-6)
`adaptive`	Logical: re-estimate kappa each iteration (default FALSE)
`step_max`	Numeric: maximum NR step size (default 5.0)
`include.mean`	Logical: include mean term (default TRUE)
`initial`	Optional initial parameter estimates
`na.action`	Function for handling missing values (default na.fail)
`verbose`	Logical: print progress information (default FALSE)

Value

An S4 object of class ARMAPMM3

See Also

arma_pmm2, arima_pmm3, lm_pmm3

Examples

```
set.seed(42)
x <- arima.sim(n = 250, list(ar = 0.7, ma = -0.3),
               innov = runif(250, -sqrt(3), sqrt(3)))
fit <- arma_pmm3(x, order = c(1, 1))
coef(fit)
```

ARPMM2-class — *S4 class for storing PMM2 AR model results*

Description

S4 class for storing PMM2 AR model results

ARPMM3-class — *S4 class for PMM3 AR model results*

Description

S4 class for PMM3 AR model results

---

ar_pmm2 — *Fit an AR model using PMM2 (wrapper)*

Description

Estimates autoregressive model parameters using the Pearson Moment Method (PMM2). PMM2 exploits third and fourth moment information to achieve more accurate parameter estimates than classical maximum likelihood, particularly for non-Gaussian innovations.

Usage

```
ar_pmm2(
  x,
  order = 1,
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification: - For AR models: a single integer (AR order) - For MA models: a single integer (MA order) - For ARMA models: vector c(p, q) (AR and MA orders) - For ARIMA models: vector c(p, d, q) (AR, differencing, and MA orders)
`method`	String: estimation method, one of "pmm2" (default), "css", "ml", "yw", "ols"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (specialized for MA/SMA models).
`max_iter`	Integer: maximum number of iterations for the algorithm
`tol`	Numeric: tolerance for convergence
`include.mean`	Logical: whether to include a mean (intercept) term
`initial`	List or vector of initial parameter estimates (optional)
`na.action`	Function for handling missing values, default is na.fail
`regularize`	Logical, add small values to diagonal for numerical stability
`reg_lambda`	Regularization parameter (if regularize=TRUE)
`verbose`	Logical: whether to print progress information

Details

PMM2 Variants:

- `"unified_global"` (default): One-step correction from MLE/CSS estimates. Fast and stable. Recommended for most AR models. Typical improvement: 3-5\
- `"unified_iterative"`: Full Newton-Raphson optimization starting from classical estimates. More accurate but computationally intensive. Best for complex models with strong non-Gaussian features.
- `"linearized"`: Uses first-order Taylor expansion. Not recommended for AR models; designed for MA/SMA where Jacobian computation is complex.

Variant Selection Guidelines:

- For AR(p) models: Use "unified_global" (default)
- If convergence issues occur: Try "unified_iterative"
- For highly skewed/heavy-tailed innovations: Use "unified_iterative"

Computational Characteristics:

- unified_global: ~2x slower than MLE (single correction step)
- unified_iterative: 5-10x slower than MLE (iterative refinement)
- linearized: ~1.5x slower than MLE (approximation-based)

Value

An S4 object of class ARPMM2 containing fitted autoregressive coefficients, residuals, central moment estimates (m2-m4), model order, intercept, original series, and convergence diagnostics.

References

Monte Carlo validation (R=50, n=200): Unified Iterative showed 2.9\ improvement over MLE for AR(1) models. See NEWS.md (Version 0.2.0) for details.

See Also

ma_pmm2, arma_pmm2, arima_pmm2

Examples

```
# Fit AR(2) model with default variant
x <- arima.sim(n = 200, list(ar = c(0.7, -0.3)))
fit1 <- ar_pmm2(x, order = 2)
coef(fit1)

# Compare variants
fit2 <- ar_pmm2(x, order = 2, pmm2_variant = "unified_iterative")
fit3 <- ar_pmm2(x, order = 2, pmm2_variant = "linearized")
```

---

ar_pmm3 *Fit an AR model using PMM3*

Description

Estimates autoregressive model parameters using the Polynomial Maximization Method of order 3 (PMM3). Designed for symmetric platykurtic innovations.

Usage

```
ar_pmm3(
  x,
  order = 1,
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Integer: AR order (default 1)
`max_iter`	Integer: maximum NR iterations (default 100)
`tol`	Numeric: convergence tolerance (default 1e-6)
`adaptive`	Logical: re-estimate kappa each iteration (default FALSE)
`step_max`	Numeric: maximum NR step size (default 5.0)
`include.mean`	Logical: include mean term (default TRUE)
`initial`	Optional initial parameter estimates
`na.action`	Function for handling missing values (default na.fail)
`verbose`	Logical: print progress information (default FALSE)

Value

An S4 object of class `ARPMM3`

See Also

`ar_pmm2`, `ma_pmm3`, `lm_pmm3`

Examples

```
set.seed(42)
x <- arima.sim(n = 200, list(ar = 0.7),
               innov = runif(200, -sqrt(3), sqrt(3)))
fit <- ar_pmm3(x, order = 1)
coef(fit)
```

`auto_mpg` *Auto MPG Dataset*

Description

Fuel consumption and vehicle characteristics for 398 automobiles from the 1970s and 1980s. This dataset is used in published PMM research to demonstrate both PMM2 (asymmetric residuals: MPG vs Weight) and PMM3 (symmetric platykurtic residuals: MPG vs Horsepower).

Usage

```
auto_mpg
```

Format

A data frame with 398 rows and 9 variables:

mpg Miles per gallon (fuel efficiency)

cylinders Number of cylinders (4, 6, or 8)

displacement Engine displacement (cubic inches)

horsepower Engine horsepower (6 missing values)

weight Vehicle weight (pounds)

acceleration Time to accelerate from 0 to 60 mph (seconds)

model_year Model year (70-82, i.e., 1970-1982)

origin Origin (1 = American, 2 = European, 3 = Japanese)

car_name Car model name

Details

Three regression examples from published PMM papers:

- **MPG vs Acceleration** (PMM2, linear): residuals have gamma3 = 0.49, g2 = 0.86 (Zabolotnii et al., 2018)
- **MPG vs Weight** (PMM2, quadratic): residuals have gamma3 = 0.8, g2 = 0.83 (Zabolotnii et al., 2025)
- **MPG vs Horsepower** (PMM3, quadratic): residuals have gamma3 ~ 0.2, gamma4 = 1.3, g3 = 0.89 (Zabolotnii et al., 2025)

Source

UCI Machine Learning Repository https://archive.ics.uci.edu/dataset/9/auto+mpg

References

Quinlan, J.R. (1993). Combining Instance-Based and Model-Based Learning. In Proceedings on the Tenth International Conference of Machine Learning, 236-243.

Zabolotnii S., Warsza Z.L., Tkachenko O. (2018) Polynomial Estimation of Linear Regression Parameters for the Asymmetric PDF of Errors. Springer AISC, vol 743. doi:10.1007/978331977179-3_75

Examples

```
data(auto_mpg)
# PMM2 example: MPG vs Acceleration (asymmetric residuals)
fit_ols <- lm(mpg ~ acceleration, data = auto_mpg)
pmm_skewness(residuals(fit_ols))  # gamma3 ~ 0.5 -> PMM2
pmm_dispatch(residuals(fit_ols))
fit_pmm2 <- lm_pmm2(mpg ~ acceleration, data = auto_mpg, na.action = na.omit)
coef(fit_pmm2)  # compare with coef(fit_ols)
```

BasePMM2-class *Base S4 class for storing PMM2 model results*

Description

Base S4 class for storing PMM2 model results

Slots

`coefficients` numeric vector of estimated parameters

`residuals` numeric vector of final residuals

`m2` numeric second central moment of initial residuals

`m3` numeric third central moment of initial residuals

`m4` numeric fourth central moment of initial residuals

`convergence` logical or integer code indicating whether algorithm converged

`iterations` numeric number of iterations performed

`call` original function call

---

coef,PMM2fit-method *Extract coefficients from PMM2fit object*

Description

Extract coefficients from PMM2fit object

Usage

```
## S4 method for signature 'PMM2fit'
coef(object, ...)
```

Arguments

`object`	PMM2fit object
`...`	Additional arguments (not used)

Value

Vector of coefficients

coef,PMM3fit-method *Extract coefficients from PMM3fit object*

Description

Extract coefficients from PMM3fit object

Usage

```
## S4 method for signature 'PMM3fit'
coef(object, ...)
```

Arguments

`object`	PMM3fit object
`...`	Additional arguments (not used)

Value

Vector of coefficients

coef,SARPMM2-method *Extract coefficients from SARPMM2 object*

Description

Extract coefficients from SARPMM2 object

Usage

```
## S4 method for signature 'SARPMM2'
coef(object, ...)
```

Arguments

`object`	SARPMM2 object
`...`	Additional arguments (not used)

Value

Named numeric vector of coefficients

coef,SMAPMM2-method *Extract coefficients from SMAPMM2 object*

Description

Extract coefficients from SMAPMM2 object

Usage

```
## S4 method for signature 'SMAPMM2'
coef(object, ...)
```

Arguments

`object`	SMAPMM2 object
`...`	Additional arguments (not used)

Value

Named vector of seasonal MA coefficients

coef,TS2fit-method *Extract coefficients from TS2fit object*

Description

Extract coefficients from TS2fit object

Usage

```
## S4 method for signature 'TS2fit'
coef(object, ...)
```

Arguments

`object`	TS2fit object
`...`	Additional arguments (not used)

Value

Named vector of coefficients

coef,TS3fit-method *Extract coefficients from TS3fit object*

Description

Extract coefficients from TS3fit object

Usage

```
## S4 method for signature 'TS3fit'
coef(object, ...)
```

Arguments

object	TS3fit object
...	Additional arguments (not used)

Value

Named vector of coefficients

compare_arima_methods *Compare ARIMA methods*

Description

Compare ARIMA methods

Usage

```
compare_arima_methods(
  x,
  order = c(1, 1, 1),
  include.mean = TRUE,
  pmm2_args = list()
)
```

Arguments

x	Numeric vector of time series data
order	Model order specification (see ts_pmm2 for format)
include.mean	Logical, whether to include intercept term
pmm2_args	List of additional arguments to pass to ts_pmm2()

Value

A named list containing the fitted objects for each estimation approach (e.g., YW/OLS/MLE or CSS/ML alongside PMM2) plus two data frames: `coefficients` (side-by-side parameter estimates) and `residual_stats` (residual RSS, MAE, skewness, and kurtosis).

compare_arma_methods *Compare ARMA methods*

Description

Compare ARMA methods

Usage

```
compare_arma_methods(
  x,
  order = c(1, 1),
  include.mean = TRUE,
  pmm2_args = list()
)
```

Arguments

x	Numeric vector of time series data
order	Model order specification (see ts_pmm2 for format)
include.mean	Logical, whether to include intercept term
pmm2_args	List of additional arguments to pass to ts_pmm2()

Value

A named list containing the fitted objects for each estimation approach (e.g., YW/OLS/MLE or CSS/ML alongside PMM2) plus two data frames: `coefficients` (side-by-side parameter estimates) and `residual_stats` (residual RSS, MAE, skewness, and kurtosis).

compare_ar_methods *Compare AR methods*

Description

Compare AR methods

Usage

```
compare_ar_methods(x, order = 1, include.mean = TRUE, pmm2_args = list())
```

Arguments

x	Numeric vector of time series data
order	Model order specification (see ts_pmm2 for format)
include.mean	Logical, whether to include intercept term
pmm2_args	List of additional arguments to pass to ts_pmm2()

Value

A named list containing the fitted objects for each estimation approach (e.g., YW/OLS/MLE or CSS/ML alongside PMM2) plus two data frames: `coefficients` (side-by-side parameter estimates) and `residual_stats` (residual RSS, MAE, skewness, and kurtosis).

`compare_ma_methods` *Compare MA methods*

Description

Compare MA methods

Usage

```
compare_ma_methods(x, order = 1, include.mean = TRUE, pmm2_args = list())
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification (see ts_pmm2 for format)
`include.mean`	Logical, whether to include intercept term
`pmm2_args`	List of additional arguments to pass to ts_pmm2()

Value

A named list containing the fitted objects for each estimation approach (e.g., YW/OLS/MLE or CSS/ML alongside PMM2) plus two data frames: `coefficients` (side-by-side parameter estimates) and `residual_stats` (residual RSS, MAE, skewness, and kurtosis).

`compare_sar_methods` *Compare SAR model estimation methods*

Description

Compares different estimation methods (OLS, PMM2, CSS, ML) for SAR models on the same data.

Usage

```
compare_sar_methods(
  x,
  order = c(1, 1),
  period = 12,
  methods = c("ols", "pmm2", "css"),
  verbose = TRUE
)
```

Arguments

x	Time series data
order	Model order c(p, P) for SAR specification
period	Seasonal period
methods	Character vector of methods to compare (default: c("ols", "pmm2", "css"))
verbose	Logical: print results to console (default TRUE)

Value

Data frame with comparison results (invisibly)

Examples

```
set.seed(42)
y <- arima.sim(n = 120,
  model = list(order = c(1, 0, 0), ar = 0.7,
    seasonal = list(order = c(1, 0, 0), ar = 0.5, period = 12)))
compare_sar_methods(y, order = c(1, 1), period = 12)
```

---

compare_ts_methods *Compare PMM2 with classical time series estimation methods*

Description

Compare PMM2 with classical time series estimation methods

Usage

```
compare_ts_methods(
  x,
  order,
  model_type = c("ar", "ma", "arma", "arima"),
  include.mean = TRUE,
  pmm2_args = list()
)
```

Arguments

x	Numeric vector of time series data
order	Model order specification (see ts_pmm2 for format)
model_type	Model type: "ar", "ma", "arma", or "arima"
include.mean	Logical, whether to include intercept term
pmm2_args	List of additional arguments to pass to ts_pmm2()

Value

A named list containing the fitted objects for each estimation approach (e.g., YW/OLS/MLE or CSS/ML alongside PMM2) plus two data frames: `coefficients` (side-by-side parameter estimates) and `residual_stats` (residual RSS, MAE, skewness, and kurtosis).

compare_with_ols — *Compare PMM2 with OLS*

Description

Compare PMM2 with OLS

Usage

```
compare_with_ols(formula, data, pmm2_args = list())
```

Arguments

formula	Model formula
data	Data frame
pmm2_args	List of arguments to pass to lm_pmm2()

Value

List with OLS and PMM2 fit objects

Examples

```
result <- compare_with_ols(mpg ~ wt, data = mtcars)
```

compute_moments — *Calculate moments and cumulants of error distribution*

Description

Calculate moments and cumulants of error distribution

Usage

```
compute_moments(errors)
```

Arguments

errors	numeric vector of errors

Value

list with moments, cumulants and theoretical variance reduction coefficient

compute_moments_pmm3 *Compute central moments for PMM3 from residuals*

Description

Computes the second, fourth, and sixth central moments, along with standardised cumulant coefficients gamma3, gamma4, gamma6, the theoretical efficiency factor g3, and the moment ratio kappa used in the PMM3 solver.

Usage

```
compute_moments_pmm3(residuals)
```

Arguments

residuals	numeric vector of residuals (typically from OLS)

Value

A list with components:

m2	Second central moment
m4	Fourth central moment
m6	Sixth central moment
gamma3	Skewness coefficient (for symmetry check)
gamma4	Excess kurtosis
gamma6	Sixth-order cumulant coefficient
g3	Theoretical variance reduction factor
kappa	Moment ratio for NR solver (NA if near-Gaussian)
improvement_pct	Expected variance reduction percentage

compute_numerical_jacobian *Unified PMM2 Estimator for Nonlinear Regression Models*

Description

This file contains a unified PMM2 implementation suitable for arbitrary nonlinear regression models (including SARIMAX) for which residuals and their derivatives (Jacobian) can be computed.

Usage

```
compute_numerical_jacobian(fn_residuals, theta, method = "Richardson")
```

Arguments

`fn_residuals`	Function(theta) returning the residual vector
`theta`	Current parameter values
`method`	Numerical differentiation method ("Richardson", "simple")

Details

Two approaches are implemented:

1. Iterative: Full iterative procedure (Nonlinear PMM2).
2. One-step (Global): Single-step correction after a classical estimator (e.g., MLE). Compute numerical Jacobian of the residual function

Uses numDeriv::jacobian to compute the derivative matrix.

Value

Jacobian matrix (n x p)

`compute_pmm2_components` *Compute PMM2 weights and components*

Description

Compute PMM2 weights and components

Usage

```
compute_pmm2_components(residuals)
```

Arguments

`residuals`	Residual vector

Value

List with moments and PMM2 parameters (gamma3, gamma4, weights)

compute_ts_residuals *Compute final residuals for time series models*

Description

Compute final residuals for time series models

Usage

```
compute_ts_residuals(coefs, model_info)
```

Arguments

coefs	Estimated coefficients
model_info	Model information

Value

Vector of residuals

create_ar_matrix *Create design matrix for AR model*

Description

Create design matrix for AR model

Usage

```
create_ar_matrix(x, p)
```

Arguments

x	Centered time series
p	AR order

Value

Design matrix with lagged values

create_residual_function

Create residual function for PMM2 optimization

Description

Create residual function for PMM2 optimization

Usage

```
create_residual_function(x, order, model_type, seasonal, include.mean)
```

Arguments

`x`	Time series data
`order`	Model order
`model_type`	Model type
`seasonal`	Seasonal specification
`include.mean`	Include intercept

Value

Function that computes residuals given parameters

create_sarma_matrix

Create design matrix for seasonal ARMA model

Description

Constructs a design matrix for Seasonal ARMA (SARMA) models, combining non-seasonal and seasonal AR and MA components.

Usage

```
create_sarma_matrix(
  x,
  residuals,
  p = 0,
  P = 0,
  q = 0,
  Q = 0,
  s = 12,
  multiplicative = FALSE
)
```

Arguments

`x`	Numeric vector (centered time series)
`residuals`	Numeric vector (initial residuals/innovations)
`p`	Non-seasonal AR order (default 0)
`P`	Seasonal AR order (default 0)
`q`	Non-seasonal MA order (default 0)
`Q`	Seasonal MA order (default 0)
`s`	Seasonal period (e.g., 12 for monthly data)
`multiplicative`	Logical, include multiplicative cross-terms (default FALSE)

Details

For a SARMA(p,q) x (P,Q)_s model:

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps} + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q} + \Theta_1 \epsilon_{t-s} + \cdots + \Theta_Q \epsilon_{t-Qs} + \epsilon_t.$$

Where:

- p, P are non-seasonal and seasonal AR orders
- q, Q are non-seasonal and seasonal MA orders
- s is the seasonal period

Value

Design matrix with lagged values. Columns are ordered as:

- First p columns: non-seasonal AR lags $(y_{t-1}, \ldots, y_{t-p})$
- Next P columns: seasonal AR lags $(y_{t-s}, \ldots, y_{t-Ps})$
- Next q columns: non-seasonal MA lags $(\epsilon_{t-1}, \ldots, \epsilon_{t-q})$
- Next Q columns: seasonal MA lags $(\epsilon_{t-s}, \ldots, \epsilon_{t-Qs})$
- If multiplicative=TRUE: AR cross-terms then MA cross-terms

Examples

```
# Simple SARMA(1,0) x (1,0)_12 model (AR+SAR, no MA)
x <- rnorm(120)
residuals <- rnorm(120)
X <- create_sarma_matrix(x, residuals, p = 1, P = 1, q = 0, Q = 0, s = 12)

# Full SARMA(1,1) x (1,1)_12 model
X <- create_sarma_matrix(x, residuals, p = 1, P = 1, q = 1, Q = 1, s = 12)
```

`create_sar_matrix` *Create design matrix for seasonal AR model*

Description

Constructs a design matrix for Seasonal Autoregressive (SAR) models, optionally including non-seasonal AR lags and multiplicative cross-terms.

Usage

```
create_sar_matrix(x, p = 0, P = 1, s = 12, multiplicative = FALSE)
```

Arguments

`x`	Numeric vector (centered time series)
`p`	Non-seasonal AR order (default 0)
`P`	Seasonal AR order (must be positive)
`s`	Seasonal period (e.g., 12 for monthly data)
`multiplicative`	Logical, include multiplicative cross-terms (default FALSE)

Details

For a SAR(P)_s model:

$$y_t = \Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps} + \epsilon_t.$$

For an additive AR(p)+SAR(P)_s model:

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps} + \epsilon_t.$$

For a multiplicative AR(p) x SAR(P)_s model (multiplicative = TRUE): Includes cross-terms such as y_{t-1-s}, y_{t-1-2s}, etc.

Value

Design matrix with lagged values. Columns are:

- First p columns: non-seasonal lags $(y_{t-1}, \ldots, y_{t-p})$
- Next P columns: seasonal lags $(y_{t-s}, \ldots, y_{t-Ps})$
- If multiplicative = TRUE: additional $p \times P$ columns for cross-terms

Examples

```
# Simple SAR(1)_12 model
x <- rnorm(120)
X <- create_sar_matrix(x, p = 0, P = 1, s = 12)

# AR(1) + SAR(1)_12 additive model
X <- create_sar_matrix(x, p = 1, P = 1, s = 12)

# AR(1) x SAR(1)_12 multiplicative model
X <- create_sar_matrix(x, p = 1, P = 1, s = 12, multiplicative = TRUE)
```

create_ts_design_matrix *Create design matrix for time series*

Description

Create design matrix for time series

Usage

```
create_ts_design_matrix(x, model_info, innovations = NULL)
```

Arguments

x	Time series data
model_info	List with model parameters
innovations	Optional innovations/residuals for MA components

Value

List with design matrix, response variable, and other components

DCOILWTICO *WTI Crude Oil Prices*

Description

Daily spot prices for West Texas Intermediate (WTI) crude oil in U.S. dollars per barrel.

Usage

```
DCOILWTICO
```

Format

A data frame with observations for each trading day:

observation_date Date of observation in YYYY-MM-DD format

DCOILWTICO Crude Oil Price: West Texas Intermediate (WTI) in USD per barrel

Source

Federal Reserve Economic Data (FRED), Federal Reserve Bank of St. Louis https://fred.stlouisfed.org/series/DCOILWTICO

Examples

```
data(DCOILWTICO)
head(DCOILWTICO)
summary(DCOILWTICO$DCOILWTICO)
```

djia2002 *Dow Jones Industrial Average Daily Data (July-December 2002)*

Description

Daily closing prices and changes of the Dow Jones Industrial Average for the second half of 2002. Used in published PMM2 research to demonstrate AR(1) estimation with asymmetric innovations.

Usage

```
djia2002
```

Format

A data frame with 127 rows and 3 variables:

date Trading date

close DJIA closing price (USD)

change Daily change in closing price (first difference; NA for the first observation)

Details

The daily changes exhibit positive skewness, making this dataset suitable for PMM2 estimation. In the original paper, an AR(1) model fitted to the change series yielded PMM2 coefficient a1 = -0.43 versus OLS a1 = -0.49, with g2 = 0.77 (23\

Source

Yahoo Finance via the quantmod R package.

References

Zabolotnii S., Tkachenko O., Warsza Z.L. (2022) Application of the Polynomial Maximization Method for Estimation Parameters of Autoregressive Models with Asymmetric Innovations. Springer AISC, vol 1427. doi:10.1007/9783031035029_37

Examples

```
data(djia2002)
# AR(1) with PMM2
changes <- na.omit(djia2002$change)
pmm_skewness(changes)  # positive skewness -> PMM2
fit <- ar_pmm2(changes, order = 1)
summary(fit)
```

estpmm_style_ma — *EstemPMM-style PMM2 estimator for MA models*

Description

EstemPMM-style PMM2 estimator for MA models

Usage

```
estpmm_style_ma(x, q = 1, include.mean = TRUE, max_iter = 50, verbose = FALSE)
```

Arguments

x	Time series
q	MA order
include.mean	Include intercept
max_iter	Maximum PMM2 iterations
verbose	Print diagnostics

Value

List with ma_coef, mean, innovations, convergence, method

estpmm_style_ma_sma — *EstemPMM-style PMM2 estimator for combined MA+SMA models*

Description

EstemPMM-style PMM2 estimator for combined MA+SMA models

Usage

```
estpmm_style_ma_sma(
  x,
  q = 1,
  Q = 1,
  s = 12,
  include.mean = TRUE,
  max_iter = 50,
  verbose = FALSE,
  multiplicative = TRUE
)
```

Arguments

`x`	Time series
`q`	MA order
`Q`	SMA order
`s`	Seasonal period
`include.mean`	Include intercept
`max_iter`	Maximum PMM2 iterations
`verbose`	Print diagnostics
`multiplicative`	Logical, if TRUE use multiplicative SARIMA model

Value

List with ma_coef, sma_coef, mean, innovations, convergence, method

---

`estpmm_style_sma` *EstemPMM-style PMM2 estimator for SMA models*

Description

EstemPMM-style PMM2 estimator for SMA models

Usage

```
estpmm_style_sma(
  x,
  Q = 1,
  s = 4,
  include.mean = TRUE,
  max_iter = 50,
  verbose = FALSE
)
```

Arguments

`x`	Time series
`Q`	SMA order
`s`	Seasonal period
`include.mean`	Include intercept
`max_iter`	Maximum PMM2 iterations
`verbose`	Print diagnostics

Value

List with sma_coef, mean, innovations, convergence, method

fitted,PMM2fit-method *Extract fitted values from PMM2fit object*

Description

Extract fitted values from PMM2fit object

Usage

```
## S4 method for signature 'PMM2fit'
fitted(object, data = NULL, ...)
```

Arguments

object	PMM2fit object
data	Optional data source for model reconstruction, if object does not contain saved data
...	Additional arguments (not used)

Value

Vector of fitted values

fitted,PMM3fit-method *Extract fitted values from PMM3fit object*

Description

Extract fitted values from PMM3fit object

Usage

```
## S4 method for signature 'PMM3fit'
fitted(object, ...)
```

Arguments

object	PMM3fit object
...	Additional arguments (not used)

Value

Vector of fitted values

fitted,TS2fit-method *Extract fitted values from TS2fit object*

Description

Extract fitted values from TS2fit object

Usage

```
## S4 method for signature 'TS2fit'
fitted(object, ...)
```

Arguments

`object`	TS2fit object
`...`	Additional arguments (not used)

Value

Vector of fitted values

fitted,TS3fit-method *Extract fitted values from TS3fit object*

Description

Extract fitted values from TS3fit object

Usage

```
## S4 method for signature 'TS3fit'
fitted(object, ...)
```

Arguments

`object`	TS3fit object
`...`	Additional arguments (not used)

Value

Vector of fitted values

fitted_values *Helper function for extracting fitted values*

Description

Helper function for extracting fitted values

Usage

```
fitted_values(object, data = NULL)
```

Arguments

object	PMM2fit object

Value

Vector of fitted values

get_ar_fitted *Get fitted values for AR model*

Description

Get fitted values for AR model

Usage

```
get_ar_fitted(object)
```

Arguments

object	TS2fit object with model_type="ar"

Value

Vector of fitted values

get_classical_estimates *Get classical (MLE/CSS) estimates for initialization*

Description

Get classical (MLE/CSS) estimates for initialization

Usage

```
get_classical_estimates(x, order, model_type, seasonal, include.mean)
```

Arguments

x	Time series data
order	Model order
model_type	Model type
seasonal	Seasonal specification
include.mean	Include intercept

Value

Vector of initial parameter estimates

get_initial_estimates *Get initial parameter estimates for time series models*

Description

Get initial parameter estimates for time series models

Usage

```
get_initial_estimates(
  model_params,
  initial = NULL,
  method = "pmm2",
  verbose = FALSE
)
```

Arguments

model_params	Validated model parameters from validate_ts_parameters
initial	Optionally user-provided initial estimates
method	Estimation method
verbose	Output detailed information

Value

List containing:

`b_init`	Vector of initial AR/MA coefficients
`x_mean`	Estimated mean (if include.mean=TRUE)
`innovations`	Initial residuals/innovations
`x_centered`	Centered (or differenced + centered) series
`m2`	Second central moment of initial residuals
`m3`	Third central moment of initial residuals
`m4`	Fourth central moment of initial residuals

`get_sarimax_jacobian` *Calculate SARIMAX Jacobian (Numerical)*

Description

Calculate SARIMAX Jacobian (Numerical)

Usage

```
get_sarimax_jacobian(theta, ...)
```

Arguments

`theta`	Parameters
`...`	Arguments passed to get_sarimax_residuals

Value

Jacobian matrix (n x p)

`get_sarimax_residuals` *Calculate SARIMAX Residuals*

Description

Calculate SARIMAX Residuals

Usage

```
get_sarimax_residuals(
  theta,
  y,
  xreg = NULL,
  order = c(0, 0, 0),
  seasonal = list(order = c(0, 0, 0), period = NA),
  include.mean = TRUE
)
```

Arguments

`theta`	Combined vector of parameters (AR, MA, SAR, SMA, Intercept, Regressors)
`y`	Time series data
`xreg`	Exogenous regressors (optional)
`order`	ARIMA order c(p, d, q)
`seasonal`	Seasonal order c(P, D, Q)
`include.mean`	Boolean, whether to include mean/intercept
`period`	Seasonal period

Value

Vector of residuals

`get_yw_estimates` *Get Yule-Walker estimates for AR(p)*

Description

Get Yule-Walker estimates for AR(p)

Usage

```
get_yw_estimates(x, p)
```

Arguments

`x`	Numeric vector
`p`	Integer value of AR order

Value

Numeric vector of length p (AR coefficients)

`linearized_pmm2_wrapper` *Wrapper for linearized PMM2 approach (MA/SMA models)*

Description

Wrapper for linearized PMM2 approach (MA/SMA models)

Usage

```
linearized_pmm2_wrapper(
  x,
  order,
  model_type,
  seasonal,
  include.mean,
  max_iter,
  tol,
  verbose
)
```

Arguments

`x`	Time series data
`order`	Model order
`model_type`	Model type
`seasonal`	Seasonal specification
`include.mean`	Include intercept
`max_iter`	Maximum iterations
`tol`	Convergence tolerance
`verbose`	Print progress

Value

PMM2 estimation results

`lm_pmm2` *PMM2: Main function for PMM2 (S=2)*

Description

Fits a linear model using the Polynomial Maximization Method (order 2), which is robust to non-Gaussian errors.

Usage

```
lm_pmm2(
  formula,
  data,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  na.action = na.fail,
  weights = NULL,
  verbose = FALSE
)
```

Arguments

`formula`	R formula for the model
`data`	data.frame containing variables in the formula
`max_iter`	integer: maximum number of iterations for the algorithm
`tol`	numeric: tolerance for convergence
`regularize`	logical: add small value to diagonal for numerical stability
`reg_lambda`	numeric: regularization parameter (if regularize=TRUE)
`na.action`	function for handling missing values, default is na.fail
`weights`	optional weight vector (not yet implemented)
`verbose`	logical: whether to print progress information

Details

The PMM2 algorithm works as follows:

1. Fits ordinary least squares (OLS) regression to obtain initial estimates
2. Computes central moments (m2, m3, m4) from OLS residuals
3. Iteratively improves parameter estimates using a gradient-based approach

PMM2 is especially useful when error terms are not Gaussian.

Value

S4 object of class `PMM2fit`

Examples

```
set.seed(123)
n <- 80
x <- rnorm(n)
y <- 2 + 3 * x + rt(n, df = 3)
dat <- data.frame(y = y, x = x)

fit <- lm_pmm2(y ~ x, data = dat)
summary(fit, formula = y ~ x, data = dat)
```

`lm_pmm3` *PMM3: Fit linear model using Polynomial Maximization Method (S=3)*

Description

Fits a linear model using PMM3, which is designed for symmetric platykurtic error distributions. Uses a cubic stochastic polynomial with Newton-Raphson solver.

Usage

```
lm_pmm3(
  formula,
  data,
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

`formula`	R formula for the model
`data`	data.frame containing variables in the formula
`max_iter`	integer: maximum number of NR iterations (default 100)
`tol`	numeric: convergence tolerance (default 1e-6)
`adaptive`	logical: re-estimate kappa each iteration (default FALSE)
`step_max`	numeric: maximum NR step size (default 5.0)
`na.action`	function for handling missing values (default na.fail)
`verbose`	logical: print progress information (default FALSE)

Details

The PMM3 algorithm works as follows:

1. Fits OLS regression to obtain initial estimates
2. Computes central moments (m2, m4, m6) from OLS residuals
3. Checks symmetry: warns if |gamma3| > 0.3 (PMM2 may be more appropriate)
4. Computes moment ratio kappa = (m6 - 3*m4*m2) / (m4 - 3*m2^2)
5. Iterates Newton-Raphson with score Z = X'(eps*(kappa - eps^2))

PMM3 achieves lower variance than OLS when errors are symmetric and platykurtic (gamma4 < 0), with theoretical efficiency g3 = 1 - gamma4^2 / (6 + 9*gamma4 + gamma6).

Value

S4 object of class `PMM3fit`

Examples

```
set.seed(123)
n <- 100
x <- rnorm(n)
y <- 2 + 3 * x + runif(n, -sqrt(3), sqrt(3))
dat <- data.frame(y = y, x = x)

fit <- lm_pmm3(y ~ x, data = dat)
summary(fit)
```

MAPMM2-class — *S4 class for storing PMM2 MA model results*

Description

S4 class for storing PMM2 MA model results

MAPMM3-class — *S4 class for PMM3 MA model results*

Description

S4 class for PMM3 MA model results

ma_build_design — *Build design matrix for MA*

Description

Build design matrix for MA

Usage

```
ma_build_design(intercept, residuals, x, q)
```

Arguments

intercept	Intercept (mean)
residuals	Initial residuals from CSS
x	Time series
q	MA order

Value

List with X (design matrix) and y (response)

ma_compute_innovations

Compute MA innovations (forward recursion)

Description

Compute MA innovations (forward recursion)

Usage

```
ma_compute_innovations(x, theta, q)
```

Arguments

x	Time series (centered)
theta	MA coefficients
q	MA order

Value

Vector of innovations

ma_pmm2

Fit an MA model using PMM2 (wrapper)

Description

Estimates moving-average model parameters using the Pearson Moment Method (PMM2). For MA models, PMM2 can achieve substantial improvements over MLE, particularly when innovations are non-Gaussian. Monte Carlo experiments showed up to 23\ reduction for MA(1) models.

Usage

```
ma_pmm2(
  x,
  order = 1,
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification: - For AR models: a single integer (AR order) - For MA models: a single integer (MA order) - For ARMA models: vector c(p, q) (AR and MA orders) - For ARIMA models: vector c(p, d, q) (AR, differencing, and MA orders)
`method`	String: estimation method, one of "pmm2" (default), "css", "ml", "yw", "ols"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (specialized for MA/SMA models, **recommended for MA**).
`max_iter`	Integer: maximum number of iterations for the algorithm
`tol`	Numeric: tolerance for convergence
`include.mean`	Logical: whether to include a mean (intercept) term
`initial`	List or vector of initial parameter estimates (optional)
`na.action`	Function for handling missing values, default is na.fail
`regularize`	Logical, add small values to diagonal for numerical stability
`reg_lambda`	Regularization parameter (if regularize=TRUE)
`verbose`	Logical: whether to print progress information

Details

PMM2 Variants:

- `"unified_global"` (default): One-step correction from MLE/CSS estimates. Fast and reliable. Typical improvement: 15-23\
- `"unified_iterative"`: Full Newton-Raphson optimization. Slower but can achieve better accuracy for complex MA models.
- `"linearized"`: Uses first-order Taylor expansion around MLE. **Recommended for MA/SMA models** as it avoids complex Jacobian computation while maintaining accuracy. Fastest option for MA models.

Variant Selection Guidelines:

- For MA(q) models: Use `"linearized"` (fastest, MA-optimized)
- If you need maximum accuracy: Try `"unified_global"` (best MSE)
- For exploration: Compare all three variants

Computational Characteristics:

- `linearized`: ~1.5x slower than MLE (recommended)
- `unified_global`: ~2x slower than MLE (high accuracy)
- `unified_iterative`: 5-10x slower than MLE (iterative)

Why MA Models Benefit Most: MA parameter estimation from MLE has known numerical difficulties due to non-identifiability and flat likelihood regions. PMM2 uses moment constraints that better resolve these issues, leading to larger improvements than for AR models.

Value

An S4 object of class `MAPMM2` containing moving-average coefficients, residual innovations, central moments, model order, intercept, original series, and convergence diagnostics.

References

Monte Carlo validation (R=50, n=200): Unified Global showed 23.0\ improvement over MLE for MA(1) models - the largest improvement among all model types. See NEWS.md (Version 0.2.0) for full comparison.

See Also

ar_pmm2, arma_pmm2, sma_pmm2

Examples

```
# Fit MA(1) model with linearized variant (recommended)
x <- arima.sim(n = 200, list(ma = 0.6))
fit1 <- ma_pmm2(x, order = 1, pmm2_variant = "linearized")
coef(fit1)

# Compare with unified_global (best accuracy)
fit2 <- ma_pmm2(x, order = 1, pmm2_variant = "unified_global")

# Higher-order MA
x2 <- arima.sim(n = 300, list(ma = c(0.7, -0.4, 0.2)))
fit3 <- ma_pmm2(x2, order = 3, pmm2_variant = "linearized")
```

---

ma_pmm3 *Fit an MA model using PMM3*

Description

Estimates moving-average model parameters using PMM3.

Usage

```
ma_pmm3(
  x,
  order = 1,
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  include.mean = TRUE,
  initial = NULL,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

x	Numeric vector of time series data
order	Integer: AR order (default 1)
max_iter	Integer: maximum NR iterations (default 100)

tol	Numeric: convergence tolerance (default 1e-6)
adaptive	Logical: re-estimate kappa each iteration (default FALSE)
step_max	Numeric: maximum NR step size (default 5.0)
include.mean	Logical: include mean term (default TRUE)
initial	Optional initial parameter estimates
na.action	Function for handling missing values (default na.fail)
verbose	Logical: print progress information (default FALSE)

Value

An S4 object of class MAPMM3

See Also

ma_pmm2, ar_pmm3, lm_pmm3

Examples

```
set.seed(42)
x <- arima.sim(n = 200, list(ma = 0.6),
               innov = runif(200, -sqrt(3), sqrt(3)))
fit <- ma_pmm3(x, order = 1)
coef(fit)
```

---

ma_sma_build_design *Build design matrix for combined MA+SMA model*

Description

Build design matrix for combined MA+SMA model

Usage

```
ma_sma_build_design(intercept, residuals, x, q, Q, s, multiplicative = TRUE)
```

Arguments

intercept	Intercept (mean)
residuals	Initial residuals from CSS
x	Time series
q	MA order
Q	SMA order
s	Seasonal period
multiplicative	Logical, if TRUE use multiplicative SARIMA model

Value

List with X (design matrix) and y (response)

ma_sma_compute_innovations *Compute combined MA+SMA innovations (forward recursion)*

Description

Compute combined MA+SMA innovations (forward recursion)

Usage

```
ma_sma_compute_innovations(
  x,
  theta_ma,
  theta_sma,
  q,
  Q,
  s,
  multiplicative = TRUE
)
```

Arguments

x	Time series (centered)
theta_ma	MA coefficients (length q)
theta_sma	SMA coefficients (length Q)
q	MA order
Q	SMA order
s	Seasonal period
multiplicative	Logical, if TRUE use multiplicative SARIMA model

Value

Vector of innovations

ma_solve_pmm2 *PMM2 solver (EstemPMM formula)*

Description

PMM2 solver (EstemPMM formula)

Usage

```
ma_solve_pmm2(
  b_init,
  X,
  Y,
  m2,
  m3,
  m4,
  max_iter = 50,
  tol = 1e-06,
  verbose = FALSE
)
```

Arguments

`b_init`	Initial coefficients vector (intercept and MA coefficients)
`X`	Design matrix
`Y`	Response vector
`m2`	Second central moment
`m3`	Third central moment
`m4`	Fourth central moment
`max_iter`	Maximum iterations
`tol`	Convergence tolerance
`verbose`	Print diagnostics

Value

List with coefficients, convergence, iterations

`optimized_direct_pmm2` *Optimized Direct Nonlinear PMM2 (DEPRECATED)*

Description

DEPRECATED: This function has been deprecated and should not be used.

Reason for Deprecation: Monte Carlo experiments (R=50, n=200) showed this direct optimization approach performs catastrophically worse than classical methods:

- AR(1): 17.0x worse MSE than MLE (3.67 vs 0.22)
- MA(1): Failed to converge in most replications
- SARIMA: Severe numerical instability

Recommended Alternatives: Use the Unified PMM2 framework instead, which showed 3-23\

- `ar_pmm2` with `pmm2_variant = "unified_global"` (default)
- `arima_pmm2` with `pmm2_variant = "unified_iterative"`
- `ma_pmm2` with `pmm2_variant = "linearized"`

Usage

```
optimized_direct_pmm2(
  theta_init,
  y,
  order = c(0, 0, 0),
  seasonal = list(order = c(0, 0, 0), period = NA)
)
```

Arguments

`theta_init`	Initial parameters
`y`	Time series
`order`	ARIMA order
`seasonal`	Seasonal order

Details

This function attempted to directly maximize the PMM2 objective function:

$$J(\theta) = \gamma_3^2/(2 + \gamma_4)$$

However, this objective surface is highly non-convex with many local optima and flat regions, making direct optimization unreliable. The Unified PMM2 approach (global/iterative corrections from classical estimates) proved far more stable and accurate.

Technical Issues:

- Objective function has flat regions near optimal values
- Gradient becomes numerically unstable when $\gamma_4 \approx -2$
- Sensitive to initialization (even from MLE estimates)
- No guarantee of convergence to global optimum

Value

This function will throw an error directing users to unified methods

Optimization result

References

Monte Carlo comparison results documented in NEWS.md (Version 0.2.0)

See Also

`ar_pmm2`, `ma_pmm2`, `arima_pmm2`

Optimized Direct Nonlinear PMM2

Uses analytical gradients to maximize the PMM2 objective function. J(theta) = gamma3^2 / (2 + gamma4)

plot,PMM2fit,missing-method

Plot diagnostic plots for PMM2fit object

Description

Plot diagnostic plots for PMM2fit object

Usage

```
## S4 method for signature 'PMM2fit,missing'
plot(x, y, which = 1:4, ...)
```

Arguments

x	PMM2fit object
y	Not used (compatibility with generic)
which	Set of plots to display (values 1-4)
...	Additional arguments passed to plotting functions

Value

Invisibly returns the input object

plot,PMM3fit,missing-method

Plot diagnostic plots for PMM3fit object

Description

Plot diagnostic plots for PMM3fit object

Usage

```
## S4 method for signature 'PMM3fit,missing'
plot(x, y, which = 1:4, ...)
```

Arguments

x	PMM3fit object
y	Not used (compatibility with generic)
which	Set of plots to display (values 1-4)
...	Additional arguments passed to plotting functions

Value

Invisibly returns the input object

plot,TS2fit,missing-method

Build diagnostic plots for TS2fit objects

Description

Build diagnostic plots for TS2fit objects

Usage

```
## S4 method for signature 'TS2fit,missing'
plot(x, y, which = c(1:4), ...)
```

Arguments

x	TS2fit object
y	Not used (for S4 method compatibility)
which	Integer vector indicating which plots to produce
...	additional arguments passed to plot functions

Value

Invisibly returns x

plot,TS3fit,missing-method

Plot diagnostic plots for TS3fit object

Description

Plot diagnostic plots for TS3fit object

Usage

```
## S4 method for signature 'TS3fit,missing'
plot(x, y, which = 1:4, ...)
```

Arguments

x	TS3fit object
y	Not used
which	Set of plots to display (values 1-4)
...	Additional arguments

Value

Invisibly returns the input object

plot_pmm2_bootstrap *Plot bootstrap distributions for PMM2 fit*

Description

Plot bootstrap distributions for PMM2 fit

Usage

```
plot_pmm2_bootstrap(object, coefficients = NULL)
```

Arguments

`object`	Result from pmm2_inference
`coefficients`	Which coefficients to plot, defaults to all

Value

Invisibly returns histogram information

PMM2fit-class *S4 class for storing PMM2 regression model results*

Description

Class for storing results of linear model estimation using PMM2

Slots

`coefficients` numeric vector of estimated parameters
`residuals` numeric vector of final residuals
`m2` numeric second central moment of initial residuals
`m3` numeric third central moment of initial residuals
`m4` numeric fourth central moment of initial residuals
`convergence` logical or integer code indicating whether algorithm converged
`iterations` numeric number of iterations performed
`call` original function call

Slots

coefficients Estimated coefficients
residuals Final residuals
m2 Second central moment
m3 Third central moment
m4 Fourth central moment
convergence Convergence status
iterations Number of iterations performed
call Original call

`pmm2_algorithm` *Universal PMM2 algorithm for all model types*

Description

Universal PMM2 algorithm for all model types

Usage

```
pmm2_algorithm(
  b_init,
  X,
  y,
  m2,
  m3,
  m4,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE,
  poly_terms = NULL
)
```

Arguments

`b_init`	Initial parameter estimates
`X`	Design matrix
`y`	Response vector
`m2`, `m3`, `m4`	Central moments
`max_iter`	Maximum number of iterations
`tol`	Tolerance for convergence
`regularize`	Whether to add regularization
`reg_lambda`	Regularization parameter
`verbose`	Whether to print progress information
`poly_terms`	Pre-computed polynomial coefficients (list with elements `A`, `B`, `C`); allows passing custom values for special scenarios, otherwise they are computed from moments

Value

List with estimation results

pmm2_inference *Bootstrap inference for PMM2 fit*

Description

Bootstrap inference for PMM2 fit

Usage

```
pmm2_inference(
  object,
  formula,
  data,
  B = 200,
  seed = NULL,
  parallel = FALSE,
  cores = NULL
)
```

Arguments

object	object of class PMM2fit
formula	the same formula that was used initially
data	data frame that was used initially
B	number of bootstrap replications
seed	(optional) for reproducibility
parallel	logical, whether to use parallel computing
cores	number of cores to use for parallel computing, defaults to auto-detect

Value

data.frame with columns: Estimate, Std.Error, t.value, p.value

pmm2_monte_carlo_compare *Monte Carlo comparison of PMM2 estimation methods*

Description

Function generates time series for given models, repeatedly estimates parameters using different methods and compares their accuracy by MSE criterion. Additionally outputs theoretical and empirical characteristics of the innovation distribution (skewness, excess kurtosis, theoretical gain of PMM2).

Usage

```
pmm2_monte_carlo_compare(
  model_specs,
  methods = c("css", "pmm2"),
  n,
  n_sim,
  innovations = list(type = "gaussian"),
  seed = NULL,
  include.mean = TRUE,
  progress = interactive(),
  verbose = FALSE
)
```

Arguments

`model_specs` List of model specifications. Each element must contain:

- **model** "ar", "ma" or "arma"
- **order** order (for AR/MA) or vector c(p, q) for ARMA
- **theta** numeric vector of true parameters; for ARMA a list `list(ar = ..., ma = ...)`
- **label** (optional) model name in report
- **innovations** (optional) description of innovation distribution: `list(type = "gamma", shape = 2)`, `list(type = "student_t", df = 5)`, etc. Can also pass an arbitrary generation function via `generator`.

`methods` Vector of estimation methods (e.g., `c("css","pmm2")`). The first method is considered baseline for relative MSE calculation.

`n` Sample size for simulation.

`n_sim` Number of Monte Carlo experiments.

`innovations` Function or distribution description, used by default for all models (if not specified in spec).

`seed` Initial seed for random number generator (optional).

`include.mean` Logical flag: whether to include intercept during estimation.

`progress` Logical flag: print Monte Carlo progress.

`verbose` Whether to print diagnostic messages on failures.

Value

List with three components:

parameter_results MSE and relative MSE for each parameter

summary Averaged MSE over parameters for each model/method

gain Comparison of theoretical and empirical PMM2 gain

pmm2_nonlinear_iterative

Universal PMM2 estimator (Iterative)

Description

Universal PMM2 estimator (Iterative)

Usage

```
pmm2_nonlinear_iterative(
  theta_init,
  fn_residuals,
  fn_jacobian = NULL,
  max_iter = 100,
  tol = 1e-06,
  verbose = FALSE
)
```

Arguments

`theta_init`	Initial parameter values
`fn_residuals`	Function(theta) returning the residual vector
`fn_jacobian`	Function(theta) returning the Jacobian matrix (n x p). J_ij = d(y_hat_i)/d(theta_j) = -d(epsilon_i)/d(theta_j). If NULL, numerical Jacobian via numDeriv is used
`max_iter`	Maximum number of iterations
`tol`	Convergence tolerance
`verbose`	Print progress

Value

List with results (theta, residuals, convergence, etc.)

pmm2_nonlinear_onestep

Universal PMM2 estimator (One-step Global)

Description

Applies a single PMM2 correction to classical estimation results.

Usage

```
pmm2_nonlinear_onestep(
  theta_classical,
  fn_residuals,
  fn_jacobian = NULL,
  verbose = FALSE
)
```

Arguments

`theta_classical`	Parameter estimates from a classical method (e.g., MLE)
`fn_residuals`	Function(theta) returning the residual vector
`fn_jacobian`	Function(theta) returning the Jacobian matrix. If NULL, numerical Jacobian via numDeriv is used
`verbose`	Print progress

Value

List with results (theta, residuals, etc.)

`pmm2_variance_factor` *Calculate theoretical skewness, kurtosis coefficients and variance reduction factor*

Description

Calculate theoretical skewness, kurtosis coefficients and variance reduction factor

Usage

```
pmm2_variance_factor(m2, m3, m4)
```

Arguments

`m2`, `m3`, `m4`	central moments of second, third and fourth orders

Value

List with fields `c3`, `c4` and `g`

`pmm2_variance_matrices` *Calculate theoretical variance matrices for OLS and PMM2*

Description

Calculate theoretical variance matrices for OLS and PMM2

Usage

```
pmm2_variance_matrices(X, m2, m3, m4)
```

Arguments

`X`	Design matrix with column of ones
`m2`, `m3`, `m4`	central moments of OLS residuals

Value

List with fields `ols`, `pmm2`, `c3`, `c4`, `g`

PMM3fit-class — *S4 class for storing PMM3 regression model results*

Description

PMM3 (S=3) is designed for symmetric platykurtic error distributions. This class is fully standalone and does NOT inherit from BasePMM2.

Slots

`coefficients` numeric vector of estimated parameters

`residuals` numeric vector of final residuals

`m2` numeric second central moment of initial residuals

`m4` numeric fourth central moment of initial residuals

`m6` numeric sixth central moment of initial residuals

`gamma4` numeric excess kurtosis coefficient

`gamma6` numeric sixth-order cumulant coefficient

`g_coefficient` numeric theoretical variance reduction factor g3

`kappa` numeric moment ratio used in NR solver

`convergence` logical indicating whether algorithm converged

`iterations` numeric number of iterations performed

`call` original function call

pmm3_variance_factor — *Calculate PMM3 theoretical variance reduction factor*

Description

Computes the standardised cumulant coefficients gamma4 and gamma6 from the raw central moments, and derives the PMM3 efficiency factor $g_3 = 1 - \gamma_4^2/(6 + 9\gamma_4 + \gamma_6)$.

Usage

```
pmm3_variance_factor(m2, m4, m6)
```

Arguments

`m2`	Second central moment
`m4`	Fourth central moment
`m6`	Sixth central moment

Value

A list with components:

`gamma4`	Excess kurtosis
`gamma6`	Sixth-order cumulant coefficient
`g3`	Theoretical variance reduction factor

`pmm_dispatch` *Automatic PMM method selection*

Description

Analyses OLS residual cumulants to recommend the best estimation method: OLS (Gaussian errors), PMM2 (asymmetric errors), or PMM3 (symmetric platykurtic errors).

Usage

```
pmm_dispatch(
  residuals,
  symmetry_threshold = 0.3,
  kurtosis_threshold = -0.7,
  g2_threshold = 0.95,
  verbose = TRUE
)
```

Arguments

`residuals`	numeric vector of OLS residuals
`symmetry_threshold`	numeric: |gamma3| threshold for symmetry (default 0.3)
`kurtosis_threshold`	numeric: gamma4 threshold for PMM3 (default -0.7)
`g2_threshold`	numeric: minimum g2 improvement to justify PMM2 (default 0.95)
`verbose`	logical: print decision reasoning (default TRUE)

Value

A list with components:

`method`	Character: "OLS", "PMM2", or "PMM3"
`gamma3`	Sample skewness
`gamma4`	Sample excess kurtosis
`gamma6`	Sample 6th cumulant coefficient
`g2`	PMM2 efficiency factor
`g3`	PMM3 efficiency factor
`g_selected`	Efficiency factor for chosen method
`improvement_pct`	Expected variance reduction percentage
`reasoning`	Human-readable decision explanation
`n`	Sample size

Examples

```
set.seed(42)
x <- rnorm(200); eps <- runif(200, -1, 1)
y <- 1 + 2 * x + eps
fit_ols <- lm(y ~ x)
pmm_dispatch(residuals(fit_ols))
```

---

pmm_gamma6 *Compute sixth-order cumulant coefficient gamma6*

Description

Calculates $\gamma_6 = m_6/m_2^3 - 15m_4/m_2^2 + 30$ from a numeric vector. For a Gaussian distribution gamma6 equals zero.

Usage

```
pmm_gamma6(x)
```

Arguments

x	numeric vector

Value

Numeric scalar: the sixth-order cumulant coefficient

---

pmm_kurtosis *Calculate kurtosis from data*

Description

Calculate kurtosis from data

Usage

```
pmm_kurtosis(x, excess = TRUE)
```

Arguments

x	numeric vector
excess	logical, whether to return excess kurtosis (kurtosis - 3)

Value

Kurtosis value

pmm_skewness *Calculate skewness from data*

Description

Calculate skewness from data

Usage

```
pmm_skewness(x)
```

Arguments

x	numeric vector

Value

Skewness value

predict,PMM2fit-method *Prediction method for PMM2fit objects*

Description

Computes predictions for new data using a fitted PMM2 model. The method extracts the formula from the fitted model, constructs a design matrix from the new data, and computes predictions via matrix multiplication. This approach works with arbitrary variable names and model specifications.

Usage

```
## S4 method for signature 'PMM2fit'
predict(object, newdata = NULL, debug = FALSE, ...)
```

Arguments

object	PMM2fit object returned by `lm_pmm2()`
newdata	Data frame containing the predictor variables with the same names as those used in the original model fit. Required parameter.
debug	Logical value indicating whether to output debug information about the prediction process. Default is FALSE.
...	Additional arguments (currently not used)

Details

The prediction is computed as `X %*% beta` where X is the design matrix constructed from `newdata` using the model formula, and beta are the estimated coefficients. The method automatically handles:

- Models with intercepts and without
- Arbitrary variable names (not limited to "x1", "x2", etc.)
- Interaction terms and transformations specified in the formula
- Automatic coefficient reordering to match design matrix columns

Value

Numeric vector of predicted values for the observations in `newdata`

Examples

```
# Fit model
fit <- lm_pmm2(mpg ~ wt + hp, data = mtcars)

# Predict on new data
newdata <- data.frame(wt = c(2.5, 3.0), hp = c(100, 150))
predictions <- predict(fit, newdata = newdata)
```

`predict,PMM3fit-method` *Predict method for PMM3fit objects*

Description

Predict method for PMM3fit objects

Usage

```
## S4 method for signature 'PMM3fit'
predict(object, newdata = NULL, ...)
```

Arguments

`object`	PMM3fit object
`newdata`	Data frame with predictor variables
`...`	Additional arguments (not used)

Value

Numeric vector of predicted values

`predict,TS2fit-method` *Prediction method for TS2fit objects*

Description

Prediction method for TS2fit objects

Usage

```
## S4 method for signature 'TS2fit'
predict(object, n.ahead = 1, ...)
```

Arguments

`object`	TS2fit object
`n.ahead`	Number of steps ahead for prediction
`...`	additional arguments (not used)

Value

Vector or list of predictions, depending on model type

`predict,TS3fit-method` *Predict method for TS3fit objects*

Description

Predict method for TS3fit objects

Usage

```
## S4 method for signature 'TS3fit'
predict(object, n.ahead = 1, ...)
```

Arguments

`object`	TS3fit object
`n.ahead`	Integer: number of steps ahead to forecast
`...`	Additional arguments (not used)

Value

Numeric vector of predicted values

`residuals,PMM2fit-method` *Extract residuals from PMM2fit object*

Description

Extract residuals from PMM2fit object

Usage

```
## S4 method for signature 'PMM2fit'
residuals(object, ...)
```

Arguments

`object`	PMM2fit object
`...`	Additional arguments (not used)

Value

Vector of residuals

residuals,PMM3fit-method

Extract residuals from PMM3fit object

Description

Extract residuals from PMM3fit object

Usage

```
## S4 method for signature 'PMM3fit'
residuals(object, ...)
```

Arguments

`object`	PMM3fit object
`...`	Additional arguments (not used)

Value

Vector of residuals

residuals,TS2fit-method

Extract residuals from TS2fit object

Description

Extract residuals from TS2fit object

Usage

```
## S4 method for signature 'TS2fit'
residuals(object, ...)
```

Arguments

`object`	TS2fit object
`...`	Additional arguments (not used)

Value

Vector of residuals (innovations)

residuals,TS3fit-method

Extract residuals from TS3fit object

Description

Extract residuals from TS3fit object

Usage

```
## S4 method for signature 'TS3fit'
residuals(object, ...)
```

Arguments

`object`	TS3fit object
`...`	Additional arguments (not used)

Value

Vector of residuals

SARIMAPMM2-class

S4 class for Seasonal ARIMA model results with PMM2

Description

This class stores the results of fitting a Seasonal Autoregressive Integrated Moving Average (SARIMA) model using the PMM2 method. It extends SARMA with differencing operators.

Details

The SARIMAPMM2 class represents fitted SARIMA(p,d,q) x (P,D,Q)_s models:

$$(1-\phi_1 B-\cdots-\phi_p B^p)(1-\Phi_1 B^s-\cdots-\Phi_P B^{Ps})(1-B)^d(1-B^s)^D y_t = (1+\theta_1 B+\cdots+\theta_q B^q)(1+\Theta_1 B^s+\cdots+\Theta_Q B^{Qs}$$

Where B is the backshift operator.

Slots

`coefficients` Numeric vector of estimated parameters

`residuals` Numeric vector of residuals/innovations

`m2` Second central moment of residuals

`m3` Third central moment of residuals

`m4` Fourth central moment of residuals

`convergence` Logical, whether PMM2 algorithm converged

`iterations` Integer, number of iterations performed

`call` Original function call

`model_type` Character, model type identifier ("sarima")

`intercept` Numeric, intercept/mean term

`original_series` Numeric vector, original time series data

`order` List with model specification: list(ar, sar, ma, sma, d, D, period)

- ar: Non-seasonal AR order (p)
- sar: Seasonal AR order (P)
- ma: Non-seasonal MA order (q)
- sma: Seasonal MA order (Q)
- d: Non-seasonal differencing order
- D: Seasonal differencing order
- period: Seasonal period (s)

See Also

`sarima_pmm2` for fitting SARIMA models

`sarima_pmm2` *Fit a Seasonal ARIMA model using PMM2 method*

Description

Fits a Seasonal Autoregressive Integrated Moving Average (SARIMA) model with both non-seasonal and seasonal differencing operators.

Usage

```
sarima_pmm2(
  x,
  order = c(0, 1, 0, 1),
  seasonal = list(order = c(1, 1), period = 12),
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  include.mean = NULL,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  ma_method = c("mle", "pmm2"),
  verbose = FALSE,
  multiplicative = TRUE
)
```

Arguments

`x` Numeric vector of time series data

`order` Vector of length 4: c(p, P, q, Q) where:

- p: Non-seasonal AR order

- P: Seasonal AR order
- q: Non-seasonal MA order
- Q: Seasonal MA order

`seasonal` List with seasonal specification: list(order = c(d, D), period = s)

- d: Non-seasonal differencing order
- D: Seasonal differencing order
- period: Seasonal period (s)

`method` Estimation method: "pmm2" (default), "css-ml", "css"

`pmm2_variant` Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson, **recommended for SARIMA**), or `"linearized"` (specialized for MA/SMA models).

`include.mean` Logical, include drift term (default TRUE for d+D=0, FALSE otherwise)

`max_iter` Maximum iterations for PMM2 algorithm (default 50)

`tol` Convergence tolerance (default 1e-6)

`regularize` Logical, use regularization for numerical stability (default TRUE)

`reg_lambda` Regularization parameter (default 1e-8)

`ma_method` Method for MA/SMA estimation: "mle" (default) or "pmm2"

`verbose` Logical, print progress information (default FALSE)

`multiplicative` Logical, use multiplicative seasonal model form with cross-terms between non-seasonal and seasonal components (default TRUE). If FALSE, uses additive form.

Details

The SARIMA(p,d,q) x (P,D,Q)_s model satisfies

$$(1-\phi_1 B-\cdots-\phi_p B^p)(1-\Phi_1 B^s-\cdots-\Phi_P B^{Ps})(1-B)^d(1-B^s)^D y_t = (1+\theta_1 B+\cdots+\theta_q B^q)(1+\Theta_1 B^s+\cdots+\Theta_Q B^{Qs}$$

Where B is the backshift operator. The model combines:

- Non-seasonal differencing: (1-B)^d
- Seasonal differencing: (1-B^s)^D
- Non-seasonal ARMA components
- Seasonal ARMA components

Variant Recommendations for SARIMA:

- `"unified_iterative"` (recommended): Monte Carlo experiments showed 16.4\ seasonal dynamics
- `"unified_global"` (default): Faster alternative with good accuracy
- `"linearized"`: Not recommended for SARIMA (designed for MA/SMA)

Value

S4 object of class SARIMAPMM2 containing:

- coefficients: Estimated parameters
- residuals: Model residuals/innovations (padded with zeros to match original length)
- m2, m3, m4: Central moments of residuals
- convergence: Convergence status
- iterations: Number of iterations performed

References

Monte Carlo validation (R=50, n=200): Unified Iterative achieved 16.4\ improvement for SARIMA models. See NEWS.md (Version 0.2.0).

See Also

sarma_pmm2, arima_pmm2

Examples

```
set.seed(123)
n <- 200
y <- arima.sim(n = n,
  model = list(order = c(1, 0, 1), ar = 0.5, ma = 0.3,
    seasonal = list(order = c(1, 0, 0), ar = 0.4, period = 12)))
fit <- sarima_pmm2(y,
  order = c(1, 0, 1, 0),
  seasonal = list(order = c(1, 0), period = 12))
summary(fit)
```

SARMAPMM2-class *S4 class for Seasonal ARMA model results with PMM2*

Description

This class stores the results of fitting a Seasonal Autoregressive Moving Average (SARMA) model using the PMM2 method. It combines both seasonal AR and seasonal MA components.

Details

The SARMAPMM2 class represents fitted SARMA models combining:

- AR(p): $\phi_1 y_{t-1} + \cdots + \phi_p y_{t-p}$
- Seasonal AR component: $\Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps}$
- MA(q): $\theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$
- Seasonal MA component: $\Theta_1 \epsilon_{t-s} + \cdots + \Theta_Q \epsilon_{t-Qs}$

Slots

`coefficients` Numeric vector of estimated parameters (SAR and SMA coefficients)

`residuals` Numeric vector of residuals/innovations

`m2` Second central moment of residuals

`m3` Third central moment of residuals

`m4` Fourth central moment of residuals

`convergence` Logical, whether PMM2 algorithm converged

`iterations` Integer, number of iterations performed

`call` Original function call

`model_type` Character, model type identifier ("sarma")

`intercept` Numeric, intercept/mean term

`original_series` Numeric vector, original time series data

`order` List with model specification: list(ar, sar, ma, sma, period)

- ar: Non-seasonal AR order (p)
- sar: Seasonal AR order (P)
- ma: Non-seasonal MA order (q)
- sma: Seasonal MA order (Q)
- period: Seasonal period (s)

See Also

`sarma_pmm2` for fitting SARMA models

`sarma_pmm2` *Fit a Seasonal ARMA model using PMM2 method*

Description

Fits a Seasonal Autoregressive Moving Average (SARMA) model that combines both seasonal AR and seasonal MA components, optionally with non-seasonal AR and MA terms as well.

Usage

```
sarma_pmm2(
  x,
  order = c(0, 1, 0, 1),
  season = list(period = 12),
  method = "pmm2",
  include.mean = TRUE,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Vector of length 4: c(p, P, q, Q) where: • p: Non-seasonal AR order • P: Seasonal AR order • q: Non-seasonal MA order • Q: Seasonal MA order
`season`	List with seasonal specification: list(period = s) where s is the seasonal period (e.g., 12 for monthly data)
`method`	Estimation method: "pmm2" (default), "css-ml", "css"

`include.mean`	Logical, include intercept/mean term (default TRUE)
`max_iter`	Maximum iterations for PMM2 algorithm (default 50)
`tol`	Convergence tolerance (default 1e-6)
`regularize`	Logical, use regularization for numerical stability (default TRUE)
`reg_lambda`	Regularization parameter (default 1e-8)
`verbose`	Logical, print progress information (default FALSE)

Details

The SARMA model combines four components:

- AR(p): $\phi_1 y_{t-1} + \cdots + \phi_p y_{t-p}$
- Seasonal AR: $\Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps}$
- MA(q): $\theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$
- Seasonal MA: $\Theta_1 \epsilon_{t-s} + \cdots + \Theta_Q \epsilon_{t-Qs}$

The PMM2 method provides more efficient parameter estimates than ML/CSS when the innovation distribution is asymmetric (non-Gaussian).

Value

S4 object of class SARMAPMM2 containing:

- coefficients: Estimated parameters
- residuals: Model residuals/innovations (padded with zeros to match original length)
- m2, m3, m4: Central moments of residuals
- convergence: Convergence status
- iterations: Number of iterations performed

See Also

`sar_pmm2`, `sma_pmm2`, `sarima_pmm2`

Examples

```
# Generate synthetic seasonal data with SARMA structure
set.seed(123)
n <- 200
y <- arima.sim(n = n, list(
  ar = 0.5, ma = 0.3,
  seasonal = list(sar = 0.6, sma = 0.4, period = 12)
))

# Fit SARMA(1,1,1,1)_12 model with PMM2
fit <- sarma_pmm2(y, order = c(1, 1, 1, 1), season = list(period = 12))
summary(fit)

# Pure seasonal model (no non-seasonal components)
fit_pure <- sarma_pmm2(y, order = c(0, 1, 0, 1), season = list(period = 12))
```

SARPMM2-class *S4 class for Seasonal AR model results with PMM2*

Description

This class stores the results of fitting a Seasonal Autoregressive (SAR) model using the PMM2 method. It extends the TS2fit class with additional slots specific to seasonal models.

Details

The SARPMM2 class represents fitted SAR models of the form

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps} + \epsilon_t.$$

Where:

- p is the non-seasonal AR order
- P is the seasonal AR order
- s is the seasonal period

Slots

`coefficients` Numeric vector of estimated parameters (AR and SAR coefficients)

`residuals` Numeric vector of residuals/innovations

`m2` Second central moment of residuals

`m3` Third central moment of residuals

`m4` Fourth central moment of residuals

`convergence` Logical, whether PMM2 algorithm converged

`iterations` Integer, number of iterations performed

`call` Original function call

`model_type` Character, model type identifier ("sar")

`intercept` Numeric, intercept/mean term

`original_series` Numeric vector, original time series data

`order` List with model specification: list(ar, sar, period)

- ar: Non-seasonal AR order (p)
- sar: Seasonal AR order (P)
- period: Seasonal period (s)

See Also

`sar_pmm2` for fitting SAR models

sar_pmm2 *Fit Seasonal AR model using PMM2 method*

Description

Fits a Seasonal Autoregressive (SAR) model using the Polynomial Maximization Method (PMM2). The model can include both non-seasonal and seasonal AR components.

Usage

```
sar_pmm2(
  x,
  order = c(0, 1),
  season = list(period = 12),
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  include.mean = TRUE,
  multiplicative = FALSE,
  max_iter = 50,
  tol = 1e-06,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Vector of length 2: c(p, P) where: • p: Non-seasonal AR order • P: Seasonal AR order
`season`	List with seasonal specification: list(period = s) where s is the seasonal period (e.g., 12 for monthly data with annual seasonality)
`method`	Estimation method: "pmm2" (default), "ols", "css"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction from MLE/CSS), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (not recommended for SAR models).
`include.mean`	Logical, include intercept/mean term (default TRUE)
`multiplicative`	Logical, use multiplicative form with cross-terms (default FALSE)
`max_iter`	Maximum iterations for PMM2 algorithm (default 50)
`tol`	Convergence tolerance (default 1e-6)
`regularize`	Logical, use regularization for numerical stability (default TRUE)
`reg_lambda`	Regularization parameter (default 1e-8)
`verbose`	Logical, print progress information (default FALSE)

Details

The SAR model has the form

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \Phi_1 y_{t-s} + \cdots + \Phi_P y_{t-Ps} + \mu + \epsilon_t.$$

Where:

- p is the non-seasonal AR order
- P is the seasonal AR order
- s is the seasonal period
- epsilon_t are innovations (errors)

The PMM2 method provides more efficient parameter estimates than OLS when the innovation distribution is asymmetric (non-Gaussian). The expected variance reduction is given by g = 1 - c3^2 / (2 + c4), where c3 and c4 are the skewness and excess kurtosis coefficients.

Variant Recommendations for SAR:

- `"unified_global"` (default): Fast one-step correction, suitable for most SAR models
- `"unified_iterative"`: Best accuracy for complex seasonal patterns
- `"linearized"`: Not recommended for SAR (designed for MA/SMA)

Value

S4 object of class SARPMM2 containing:

- coefficients: Estimated AR and SAR parameters
- residuals: Model residuals/innovations (padded with zeros to match original length)
- m2, m3, m4: Central moments of residuals
- convergence: Convergence status
- iterations: Number of iterations performed

See Also

`ar_pmm2`, `ts_pmm2`, `compare_sar_methods`

Examples

```
# Generate synthetic seasonal data
n <- 120
y <- arima.sim(n = n, list(ar = 0.7, seasonal = list(sar = 0.5, period = 12)))

# Fit SAR(1,1)_12 model with PMM2
fit <- sar_pmm2(y, order = c(1, 1), season = list(period = 12))
summary(fit)

# Simple seasonal model (no non-seasonal component)
fit_pure_sar <- sar_pmm2(y, order = c(0, 1), season = list(period = 12))

# Compare with OLS
fit_ols <- sar_pmm2(y, order = c(1, 1), season = list(period = 12), method = "ols")
```

`SMAPMM2-class` *S4 class for Seasonal MA PMM2 results*

Description

This class stores the results of fitting a Seasonal Moving Average (SMA) model using the Polynomial Maximization Method (PMM2).

Details

The SMA(Q)_s model is expressed as

$$y_t = \mu + \epsilon_t + \Theta_1 \epsilon_{t-s} + \Theta_2 \epsilon_{t-2s} + \cdots + \Theta_Q \epsilon_{t-Qs}.$$

Where:

- Q is the seasonal MA order
- s is the seasonal period
- epsilon_t are innovations

Slots

`coefficients` Estimated seasonal MA coefficients (Theta_1, Theta_2, ..., Theta_Q)

`innovations` Estimated innovations (residuals epsilon_t)

`m2` Second central moment (variance) of innovations

`m3` Third central moment (skewness indicator) of innovations

`m4` Fourth central moment (kurtosis indicator) of innovations

`convergence` Logical indicating whether PMM2 algorithm converged

`iterations` Number of iterations required for convergence

`call` The function call that created this object

`model_type` Character string "sma"

`intercept` Model intercept (mean)

`original_series` Original time series data

`order` List with Q (seasonal MA order) and s (seasonal period)

See Also

`sma_pmm2` for fitting SMA models

sma_build_design *Build design matrix for SMA*

Description

Build design matrix for SMA

Usage

```
sma_build_design(intercept, residuals, x, Q, s)
```

Arguments

intercept	Intercept (mean)
residuals	Initial residuals from CSS
x	Time series
Q	SMA order
s	Seasonal period

Value

List with X (design matrix) and y (response)

sma_compute_innovations *Compute SMA innovations (forward recursion)*

Description

Compute SMA innovations (forward recursion)

Usage

```
sma_compute_innovations(x, Theta, Q, s)
```

Arguments

x	Time series (centered)
Theta	SMA coefficients
Q	SMA order
s	Seasonal period

Value

Vector of innovations

sma_css_fit — *CSS fit for Seasonal MA model*

Description

CSS fit for Seasonal MA model

Usage

```
sma_css_fit(x, Q, s, include_mean = TRUE, verbose = FALSE)
```

Arguments

x	Time series data
Q	Seasonal MA order
s	Seasonal period
include_mean	Include intercept
verbose	Print diagnostics

Value

List with coefficients, intercept, residuals, convergence info

sma_pmm2 — *Fit a Seasonal MA model using PMM2*

Description

Fits a Seasonal Moving Average (SMA) model using the Polynomial Maximization Method (PMM2). This is particularly effective when the innovations have asymmetric or non-Gaussian distributions.

Usage

```
sma_pmm2(
  x,
  order = 1,
  season = list(period = 12),
  method = "pmm2",
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector (time series data)
`order`	Seasonal MA order (Q)
`season`	List with seasonal specification: list(period = s)
`method`	Estimation method: "pmm2" or "css"
`pmm2_variant`	Character string specifying PMM2 implementation variant. Options: `"unified_global"` (default, one-step correction), `"unified_iterative"` (full Newton-Raphson), or `"linearized"` (specialized for MA/SMA models, **recommended for SMA**).
`max_iter`	Maximum iterations for PMM2 algorithm
`tol`	Convergence tolerance
`include.mean`	Include intercept in the model
`na.action`	Function to handle missing values
`regularize`	Add regularization to Jacobian matrix
`reg_lambda`	Regularization parameter
`verbose`	Print diagnostic information

Details

The SMA(Q)_s model has the form

$$y_t = \mu + \epsilon_t + \Theta_1 \epsilon_{t-s} + \Theta_2 \epsilon_{t-2s} + \cdots + \Theta_Q \epsilon_{t-Qs}.$$

Where:

- Q is the seasonal MA order
- s is the seasonal period (12 for monthly, 4 for quarterly)
- epsilon_t are innovations (errors)

The PMM2 method provides more efficient parameter estimates than ML/CSS when the innovation distribution is asymmetric (non-Gaussian). The expected variance reduction is given by g = 1 - c3^2 / (2 + c4), where c3 and c4 are the skewness and excess kurtosis coefficients.

Variant Recommendations for SMA:

- `"linearized"` (recommended): Fastest and most stable for SMA models, avoids complex Jacobian computation while maintaining accuracy
- `"unified_global"` (default): Good balance of speed and accuracy
- `"unified_iterative"`: Best accuracy but slower, use for complex SMA patterns

Value

An S4 object of class `SMAPMM2` containing:

- coefficients: Seasonal MA coefficients (Theta_1, Theta_2, ..., Theta_Q)
- innovations: Estimated innovations (residuals)
- m2, m3, m4: Central moments of innovations
- convergence: Convergence status
- iterations: Number of iterations performed
- intercept: Model intercept
- original_series: Original time series
- order: Model order list(Q, s)

See Also

ma_pmm2, sar_pmm2, arima_pmm2

Examples

```
# Generate synthetic seasonal data
set.seed(123)
n <- 120
s <- 12
theta <- 0.6

# Gamma innovations (asymmetric)
innov <- rgamma(n, shape = 2, scale = 1) - 2
y <- numeric(n)
for (t in 1:n) {
  ma_term <- if (t > s) theta * innov[t - s] else 0
  y[t] <- innov[t] + ma_term
}

# Fit SMA(1)_12 model with PMM2
fit <- sma_pmm2(y, order = 1, season = list(period = 12))
summary(fit)

# Compare with CSS
fit_css <- sma_pmm2(y, order = 1, season = list(period = 12), method = "css")
```

---

sma_pmm2_fit — *Fit Seasonal MA model using PMM2*

Description

Fit Seasonal MA model using PMM2

Usage

```
sma_pmm2_fit(x, Q, s, css_fit, max_iter = 50, tol = 1e-06, verbose = FALSE)
```

Arguments

x	Time series data
Q	Seasonal MA order
s	Seasonal period
css_fit	Initial CSS fit
max_iter	Maximum iterations
tol	Convergence tolerance
verbose	Print progress

Value

List with coefficients, intercept, innovations, convergence info

`solve_pmm2` *Universal solver for PMM2 system of equations*

Description

Universal solver for PMM2 system of equations

Usage

```
solve_pmm2(
  b_init,
  X,
  y,
  m2,
  m3,
  m4,
  max_iter = 1000,
  tol = 1e-05,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`b_init`	Initial parameter estimates (usually from OLS or MLE)
`X`	Design matrix (including intercept and all predictors)
`y`	Response vector
`m2`	Second central moment of residuals
`m3`	Third central moment of residuals
`m4`	Fourth central moment of residuals
`max_iter`	Maximum number of iterations
`tol`	Convergence tolerance
`regularize`	Whether to add regularization to Jacobian matrix
`reg_lambda`	Regularization parameter
`verbose`	Print progress information

Value

Vector of PMM2 parameter estimates

solve_pmm2_step *PMM2 step solver*

Description

Solves the linearized system to find the parameter update. Based on the Taylor expansion: e(theta) ~ e(theta_k) - J * delta

Usage

```
solve_pmm2_step(residuals, J, pmm_stats)
```

Arguments

residuals	Current residuals
J	Jacobian matrix (n x p), where J_ij = d(y_hat_i)/d(theta_j). We assume the standard regression definition: y = f(theta) + e, so e = y - f(theta) and d(e)/d(theta) = -d(f)/d(theta). We expect J = d(f)/d(theta) (gradient of the regression function).
pmm_stats	Statistics from compute_pmm2_components

Value

Update vector delta

summary,PMM2fit-method *Generic summary method for PMM2fit objects*

Description

Generic summary method for PMM2fit objects

Usage

```
## S4 method for signature 'PMM2fit'
summary(object, formula = NULL, data = NULL, B = 100, ...)
```

Arguments

object	object of class "PMM2fit"
formula	(optional) formula used for the model
data	(optional) data used
B	number of bootstrap replications for statistical inference
...	additional arguments (not used)

Value

Prints summary to console; returns object (invisibly).

summary,PMM3fit-method

Summary method for PMM3fit objects

Description

Summary method for PMM3fit objects

Usage

```
## S4 method for signature 'PMM3fit'
summary(object, ...)
```

Arguments

`object`	PMM3fit object
`...`	Additional arguments (not used)

Value

Prints summary to console; returns object (invisibly)

summary,SARIMAPMM2-method

Generic summary method for SARIMAPMM2 objects

Description

Generic summary method for SARIMAPMM2 objects

Usage

```
## S4 method for signature 'SARIMAPMM2'
summary(object, ...)
```

Arguments

`object`	object of class "SARIMAPMM2"
`...`	additional arguments (not used)

Value

Prints summary to console; returns object (invisibly).

summary,SARMAPMM2-method

Generic summary method for SARMAPMM2 objects

Description

Generic summary method for SARMAPMM2 objects

Usage

```
## S4 method for signature 'SARMAPMM2'
summary(object, ...)
```

Arguments

`object`	object of class "SARMAPMM2"
`...`	additional arguments (not used)

Value

Prints summary to console; returns object (invisibly).

summary,SARPMM2-method

Summary method for SARPMM2 objects

Description

Summary method for SARPMM2 objects

Usage

```
## S4 method for signature 'SARPMM2'
summary(object, ...)
```

Arguments

`object`	SARPMM2 object
`...`	Additional arguments (not used)

Value

Invisibly returns the object

summary,SMAPMM2-method

Summary method for SMAPMM2 objects

Description

Summary method for SMAPMM2 objects

Usage

```
## S4 method for signature 'SMAPMM2'
summary(object, ...)
```

Arguments

`object`	SMAPMM2 object
`...`	Additional arguments (not used)

Value

Invisibly returns the object

summary,TS2fit-method *Generic summary method for TS2fit objects*

Description

Generic summary method for TS2fit objects

Usage

```
## S4 method for signature 'TS2fit'
summary(object, ...)
```

Arguments

`object`	object of class "TS2fit" or subclass
`...`	additional arguments (not used)

Value

Prints summary to console; returns object (invisibly).

summary,TS3fit-method *Summary method for TS3fit objects*

Description

Summary method for TS3fit objects

Usage

```
## S4 method for signature 'TS3fit'
summary(object, ...)
```

Arguments

object	TS3fit object
...	Additional arguments (not used)

Value

Prints summary to console; returns object (invisibly)

test_symmetry *Test whether residuals are sufficiently symmetric for PMM3*

Description

Computes the skewness coefficient gamma3 and checks whether its absolute value falls below a given threshold. This helps decide between PMM2 (asymmetric) and PMM3 (symmetric platykurtic) estimation.

Usage

```
test_symmetry(x, threshold = 0.3)
```

Arguments

x	numeric vector of residuals
threshold	numeric threshold for |gamma3| (default 0.3)

Value

A list with components:

gamma3	Sample skewness coefficient
is_symmetric	Logical: TRUE if |gamma3| <= threshold
message	Human-readable verdict

`TS2fit-class` *Base S4 class for storing PMM2 time series model results*

Description

Base class for storing results of time series model estimation using PMM2

Slots

`coefficients` numeric vector of estimated parameters

`residuals` numeric vector of final residuals

`m2` numeric second central moment of initial residuals

`m3` numeric third central moment of initial residuals

`m4` numeric fourth central moment of initial residuals

`convergence` logical or integer code indicating whether algorithm converged

`iterations` numeric number of iterations performed

`call` original function call

`model_type` character string indicating model type

`intercept` numeric value of intercept

`original_series` numeric vector of original time series

`order` list of order parameters

Slots

coefficients Estimated coefficients

residuals Final residuals

m2 Second central moment

m3 Third central moment

m4 Fourth central moment

convergence Convergence status

iterations Number of iterations performed

call Original call

model_type Model type

intercept Intercept

original_series Original time series

order Model orders

TS3fit-class — *Base S4 class for PMM3 time series model results*

Description

Stores results from time series estimation using PMM3 (S=3). Designed for symmetric platykurtic innovations. Does NOT inherit from BasePMM2 or TS2fit.

Slots

`coefficients` numeric vector of estimated parameters

`residuals` numeric vector of final residuals/innovations

`m2` numeric second central moment of initial residuals

`m4` numeric fourth central moment of initial residuals

`m6` numeric sixth central moment of initial residuals

`gamma4` numeric excess kurtosis coefficient

`gamma6` numeric sixth-order cumulant coefficient

`g_coefficient` numeric theoretical variance reduction factor g3

`kappa` numeric moment ratio used in NR solver

`convergence` logical whether algorithm converged

`iterations` numeric number of iterations performed

`call` original function call

`model_type` character string indicating model type

`intercept` numeric intercept value

`original_series` numeric vector of original time series

`order` list of order parameters

ts_pmm2 — *Fit a time series model using the PMM2 method*

Description

Fit a time series model using the PMM2 method

Usage

```
ts_pmm2(
  x,
  order,
  model_type = c("ar", "ma", "arma", "arima"),
  method = "pmm2",
  max_iter = 50,
  tol = 1e-06,
  include.mean = TRUE,
```

```
  initial = NULL,
  na.action = na.fail,
  regularize = TRUE,
  reg_lambda = 1e-08,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification: - For AR models: a single integer (AR order) - For MA models: a single integer (MA order) - For ARMA models: vector c(p, q) (AR and MA orders) - For ARIMA models: vector c(p, d, q) (AR, differencing, and MA orders)
`model_type`	String specifying the model type: "ar", "ma", "arma", or "arima"
`method`	String: estimation method, one of "pmm2" (default), "css", "ml", "yw", "ols"
`max_iter`	Integer: maximum number of iterations for the algorithm
`tol`	Numeric: tolerance for convergence
`include.mean`	Logical: whether to include a mean (intercept) term
`initial`	List or vector of initial parameter estimates (optional)
`na.action`	Function for handling missing values, default is na.fail
`regularize`	Logical, add small values to diagonal for numerical stability
`reg_lambda`	Regularization parameter (if regularize=TRUE)
`verbose`	Logical: whether to print progress information

Details

The PMM2 algorithm works as follows:

1. Fits an initial model using a standard method (OLS, Yule-Walker, CSS or ML)
2. Computes central moments (m2, m3, m4) from initial residuals/innovations
3. Uses these moments with a specialized solver (pmm2_algorithm) to find robust parameter estimates

Value

An S4 object `TS2fit` of the corresponding subclass

`ts_pmm2_inference` *Bootstrap inference for PMM2 time series models*

Description

Bootstrap inference for PMM2 time series models

Usage

```
ts_pmm2_inference(
  object,
  x = NULL,
  B = 200,
  seed = NULL,
  block_length = NULL,
  method = c("residual", "block"),
  parallel = FALSE,
  cores = NULL,
  debug = FALSE
)
```

Arguments

object	object of class TS2fit
x	(optional) original time series; if NULL, uses object@original_series
B	number of bootstrap replications
seed	(optional) for reproducibility
block_length	block length for block bootstrap; if NULL, uses heuristic value
method	bootstrap type: "residual" or "block"
parallel	logical, whether to use parallel computing
cores	number of cores for parallel computing
debug	logical, whether to output additional diagnostic information

Value

data.frame with columns: Estimate, Std.Error, t.value, p.value

---

ts_pmm3 *Fit a time series model using PMM3*

Description

Core function that fits AR, MA, ARMA, or ARIMA models using the Polynomial Maximization Method of order 3 (PMM3). Designed for symmetric platykurtic innovations.

Usage

```
ts_pmm3(
  x,
  order,
  model_type = c("ar", "ma", "arma", "arima"),
  max_iter = 100,
  tol = 1e-06,
  adaptive = FALSE,
  step_max = 5,
  include.mean = TRUE,
```

```
  initial = NULL,
  na.action = na.fail,
  verbose = FALSE
)
```

Arguments

`x`	Numeric vector of time series data
`order`	Model order specification (see `ts_pmm2` for format)
`model_type`	Character: "ar", "ma", "arma", or "arima"
`max_iter`	Integer: maximum NR iterations (default 100)
`tol`	Numeric: convergence tolerance (default 1e-6)
`adaptive`	Logical: re-estimate kappa each iteration (default FALSE)
`step_max`	Numeric: maximum NR step size (default 5.0)
`include.mean`	Logical: include mean/intercept term (default TRUE)
`initial`	Optional initial parameter estimates
`na.action`	Function for handling missing values (default na.fail)
`verbose`	Logical: print progress information (default FALSE)

Details

The PMM3 time series algorithm:

1. Obtains initial estimates via classical methods (OLS/YW for AR, CSS for MA/ARMA/ARIMA)
2. Computes moments m2, m4, m6 and kappa from initial residuals
3. Checks symmetry: warns if |gamma3| > 0.3
4. Applies Newton-Raphson with PMM3 score equations

PMM3 is beneficial when innovations are symmetric and platykurtic (gamma4 < 0), e.g. uniform, truncated normal.

Value

An S4 object of the appropriate TS3fit subclass

`unified_pmm2_wrapper` *Unified PMM2 Wrapper for Time Series Models*

Description

Universal wrapper that dispatches to appropriate PMM2 variant based on model structure and user preference.

Usage

```
unified_pmm2_wrapper(
  x,
  order,
  model_type,
  pmm2_variant = c("unified_global", "unified_iterative", "linearized"),
  seasonal = NULL,
  include.mean = TRUE,
  max_iter = 100,
  tol = 1e-06,
  verbose = FALSE
)
```

Arguments

`x`	Time series data
`order`	Model order specification (depends on model_type)
`model_type`	Type of model: "ar", "ma", "arma", "arima", "sarima"
`pmm2_variant`	PMM2 implementation variant: • `"unified_global"` (default) - One-step correction, fast and stable • `"unified_iterative"` - Full iterative procedure for maximum accuracy • `"linearized"` - Specialized linear approach for MA/SMA models
`seasonal`	Seasonal specification for SARIMA models
`include.mean`	Include intercept/mean term
`max_iter`	Maximum iterations
`tol`	Convergence tolerance
`verbose`	Print progress information

Value

List with PMM2 estimation results

`update_ma_innovations` *Update MA model innovations*

Description

Update MA model innovations

Usage

```
update_ma_innovations(x, ma_coef)
```

Arguments

`x`	Centered time series
`ma_coef`	Vector of MA coefficients

Value

Vector of innovations

`validate_ts_parameters` *Validate and prepare time series parameters*

Description

Validate and prepare time series parameters

Usage

```
validate_ts_parameters(x, order, model_type, include.mean)
```

Arguments

`x`	Time series data
`order`	Model order specification
`model_type`	Model type (ar, ma, arma, or arima)
`include.mean`	Whether to include mean/intercept

Value

List of validated parameters and model information

Index

∗ **datasets**
auto_mpg, 23
DCOILWTICO, 38
djia2002, 39
∗ **internal**
.g2_factor, 6
.g3_factor, 6
.pmm2_fit, 7
.pmm3_nr_solver, 8
.pmm3_nr_solver_nonlinear, 8
.residual_cumulants, 9
.ts_pmm2_fit, 10
compute_numerical_jacobian, 32
compute_ts_residuals, 34
create_ar_matrix, 34
create_residual_function, 35
create_ts_design_matrix, 38
EstemPMM-package, 4
estpmm_style_ma, 40
estpmm_style_ma_sma, 40
estpmm_style_sma, 41
fitted_values, 44
get_ar_fitted, 44
get_classical_estimates, 45
get_initial_estimates, 45
get_yw_estimates, 47
linearized_pmm2_wrapper, 47
ma_build_design, 51
ma_compute_innovations, 52
ma_sma_build_design, 55
ma_sma_compute_innovations, 56
ma_solve_pmm2, 56
optimized_direct_pmm2, 57
pmm2_algorithm, 62
sma_build_design, 84
sma_compute_innovations, 84
sma_css_fit, 85
sma_pmm2_fit, 87
solve_pmm2, 88
unified_pmm2_wrapper, 98
update_ma_innovations, 99
validate_ts_parameters, 100
.g2_factor, 6
.g3_factor, 6
.pmm2_fit, 7
.pmm3_nr_solver, 8
.pmm3_nr_solver_nonlinear, 8
.residual_cumulants, 9
.ts_pmm2_fit, 10

AIC,PMM2fit-method, 11
AIC,PMM3fit-method, 11
AIC,TS3fit-method, 12
ar_pmm2, *5*, *14*, *17*, 19, *22*, *54*, *57*, *58*, *82*
ar_pmm3, *5*, 21, *55*
arima_pmm2, *5*, 13, *15*, *17*, *21*, *57*, *58*, *77*, *87*
arima_pmm3, *5*, 15, *19*
ARIMAPMM2-class, 12
ARIMAPMM3-class, 12
arma_pmm2, *5*, *14*, 16, *19*, *21*, *54*
arma_pmm3, *5*, *15*, 18
ARMAPMM2-class, 16
ARMAPMM3-class, 16
ARPMM2-class, 19
ARPMM3-class, 19
auto_mpg, 23

BasePMM2-class, 24

coef,PMM2fit-method, 24
coef,PMM3fit-method, 25
coef,SARPMM2-method, 25
coef,SMAPMM2-method, 26
coef,TS2fit-method, 26
coef,TS3fit-method, 27
compare_ar_methods, 28
compare_arima_methods, 27
compare_arma_methods, 28
compare_ma_methods, 29
compare_sar_methods, 29, *82*
compare_ts_methods, *5*, 30
compare_with_ols, *4*, 31
compute_moments, *5*, 31
compute_moments_pmm3, *5*, 32
compute_numerical_jacobian, 32
compute_pmm2_components, 33
compute_ts_residuals, 34

create_ar_matrix, 34
create_residual_function, 35
create_sar_matrix, 37
create_sarma_matrix, 35
create_ts_design_matrix, 38

DCOILWTICO, 38
djia2002, 39

EstemPMM (EstemPMM-package), 4
EstemPMM-package, 4
estpmm_style_ma, 40
estpmm_style_ma_sma, 40
estpmm_style_sma, 41

fitted,PMM2fit-method, 42
fitted,PMM3fit-method, 42
fitted,TS2fit-method, 43
fitted,TS3fit-method, 43
fitted_values, 44

get_ar_fitted, 44
get_classical_estimates, 45
get_initial_estimates, 45
get_sarimax_jacobian, 46
get_sarimax_residuals, 46
get_yw_estimates, 47

linearized_pmm2_wrapper, 47
lm_pmm2, *4*, 48
lm_pmm3, *5*, *15*, *19*, *22*, 49, *55*

ma_build_design, 51
ma_compute_innovations, 52
ma_pmm2, *5*, *17*, *21*, 52, *55*, *57*, *58*, *87*
ma_pmm3, *5*, *22*, 54
ma_sma_build_design, 55
ma_sma_compute_innovations, 56
ma_solve_pmm2, 56
MAPMM2-class, 51
MAPMM3-class, 51

optimized_direct_pmm2, 57

plot,PMM2fit,missing-method, 59
plot,PMM3fit,missing-method, 59
plot,TS2fit,missing-method, 60
plot,TS3fit,missing-method, 60
plot_pmm2_bootstrap, 61
pmm2_algorithm, 62
pmm2_inference, *5*, 63
pmm2_monte_carlo_compare, 63
pmm2_nonlinear_iterative, 65
pmm2_nonlinear_onestep, 65
pmm2_variance_factor, 66
pmm2_variance_matrices, 66
PMM2fit-class, 61
pmm3_variance_factor, *5*, 67
PMM3fit-class, 67
pmm_dispatch, *5*, 68
pmm_gamma6, *5*, 69
pmm_kurtosis, *5*, 69
pmm_skewness, *5*, 70
predict,PMM2fit-method, 70
predict,PMM3fit-method, 71
predict,TS2fit-method, 71
predict,TS3fit-method, 72

residuals,PMM2fit-method, 72
residuals,PMM3fit-method, 73
residuals,TS2fit-method, 73
residuals,TS3fit-method, 74

sar_pmm2, *79*, *80*, 81, *87*
sarima_pmm2, *14*, *75*, 75, *79*
SARIMAPMM2-class, 74
sarma_pmm2, *77*, *78*, 78
SARMAPMM2-class, 77
SARPMM2-class, 80
sma_build_design, 84
sma_compute_innovations, 84
sma_css_fit, 85
sma_pmm2, *54*, *79*, *83*, 85
sma_pmm2_fit, 87
SMAPMM2-class, 83
solve_pmm2, 88
solve_pmm2_step, 89
summary,PMM2fit-method, 89
summary,PMM3fit-method, 90
summary,SARIMAPMM2-method, 90
summary,SARMAPMM2-method, 91
summary,SARPMM2-method, 91
summary,SMAPMM2-method, 92
summary,TS2fit-method, 92
summary,TS3fit-method, 93

test_symmetry, *5*, 93
TS2fit-class, 94
TS3fit-class, 95
ts_pmm2, *5*, *82*, 95, *98*
ts_pmm2_inference, *5*, 96
ts_pmm3, *5*, 97

unified_pmm2_wrapper, 98
update_ma_innovations, 99

validate_ts_parameters, 100